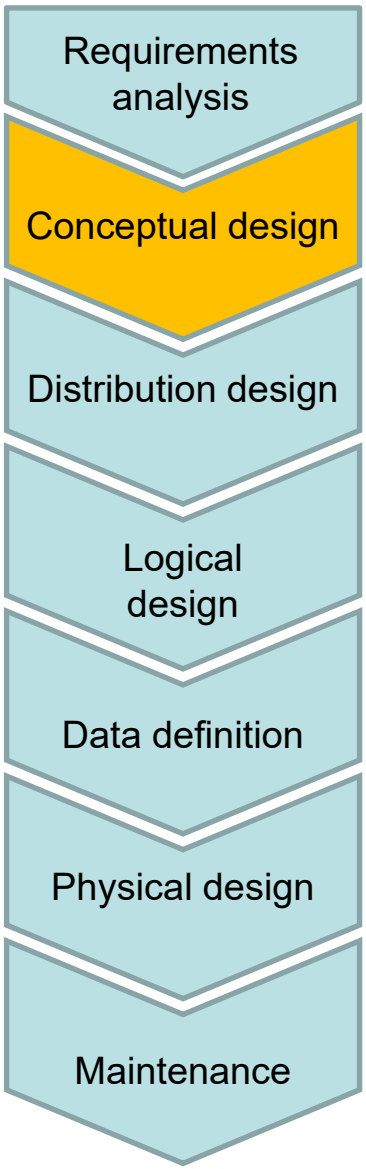
Technische Hochschule Rosenheim

Chapter 6 – Conceptual database design

Databases lectures

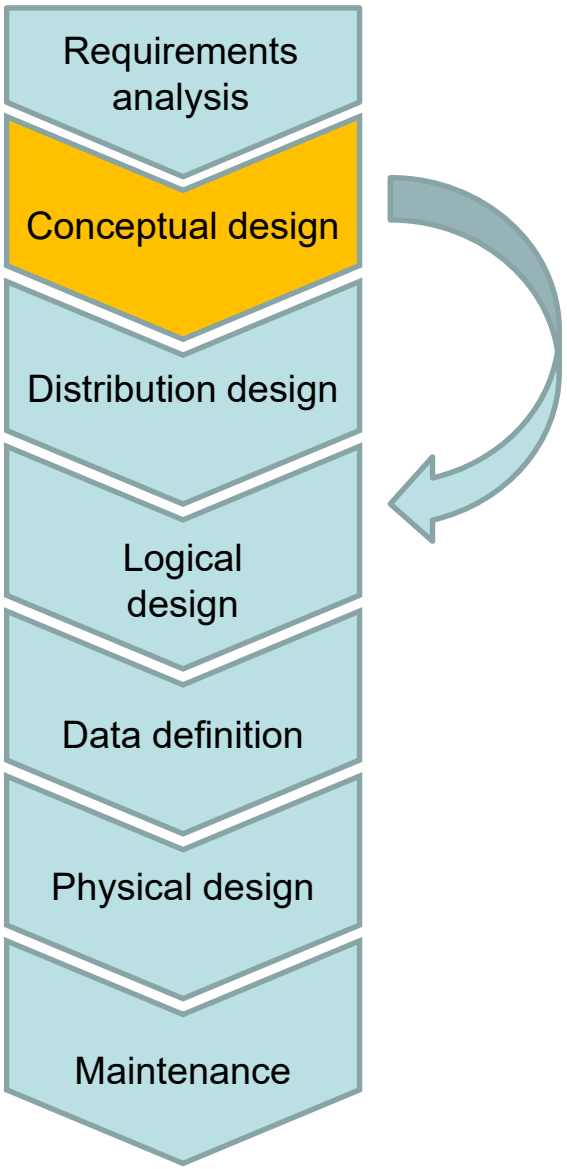
Prof. Dr Kai Höfig

Conceptual design – revision



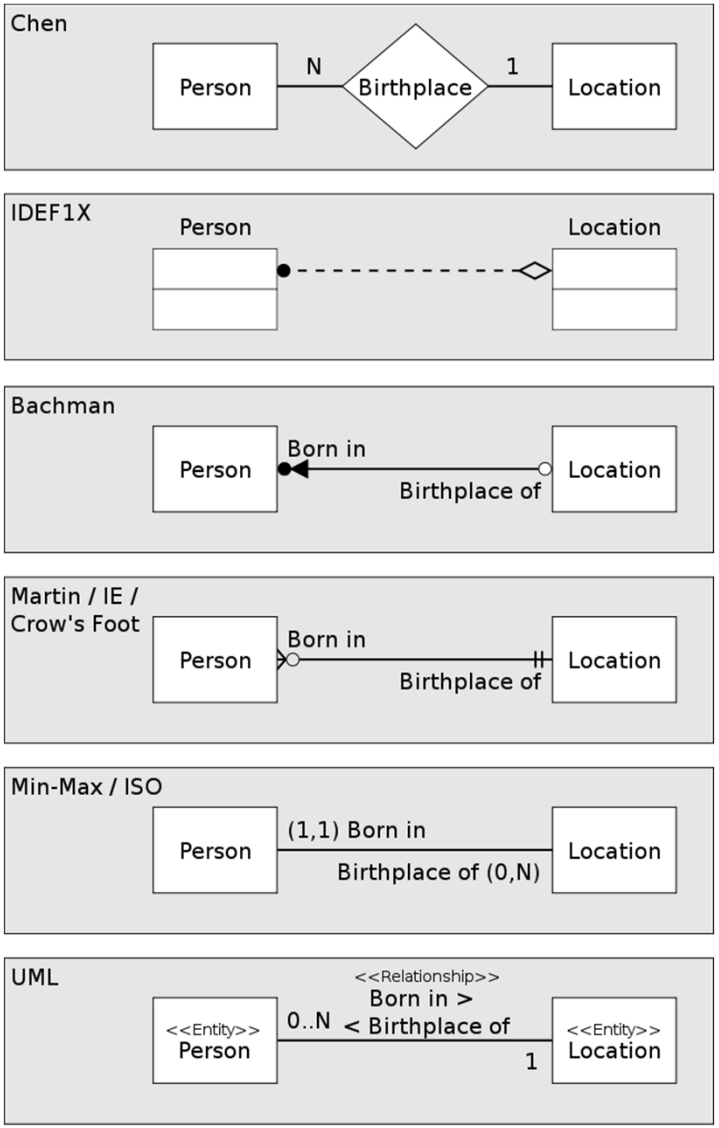
- Aim: initial formal description of the specialist problem, *regardless* of the system to be used later
- Linguistic devices: abstract (semantic) data model
- Method:
  - Modelling views, e.g. for different specialist departments
  - Analysis for conflicts
  - Integration of the views into an overall schema
- Result: conceptual data model, typically an ER or UML diagram
- Most important characteristics of design steps
  - Information preservation
    All data that could be saved in the previous model can also be saved in the new model
  - Consistency preservation
    Rules and restrictions in the previous model can also be ensured in the new model

Entity-Relationship (ER) model



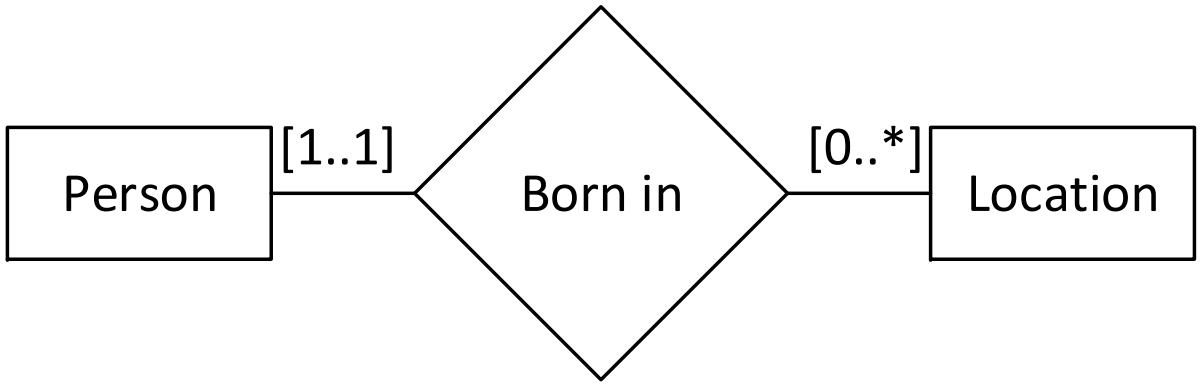
- The entity-relationship model allows us to graphically represent entities, attributes and relationships of data using simple language constructs (**syntax**).
- A relational database model can then be derived from this in a (partially) automated process (**semantics**).
- When the rational database model is drafted, we can check whether BCNF is reached.

Overview: Different ER-Diagram notations



ER-Diagrams in Minmax-Chen Notation

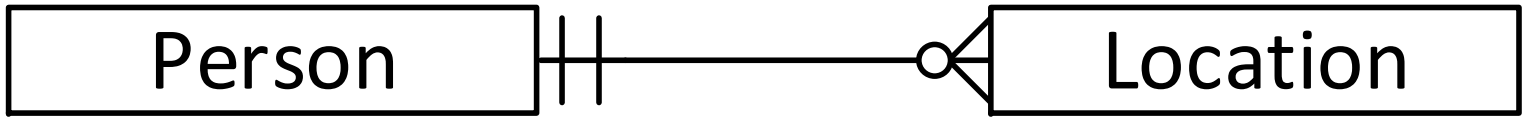
- We are using here the **Chen Notation** as the diagram type to express our **conceptual** design and we use **min-max-notation** to express quantities.
- Good to read, easy to draw, easy to comprehend by untrained persons. Used to develop or derive a logical design.



- We read here: *Every person is born in exactly one location and every location, zero or any number of persons are born.*

ER-Diagrams in Crow's Foot Notation

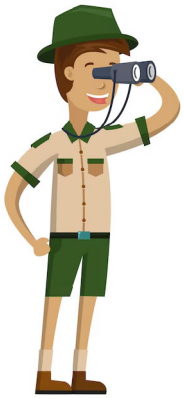
- We are using here the **Crow's Foot Notation** as the diagram type to express our **logical** design.
- Good to document actual implementations of conceptual models and often used in technical documentations.



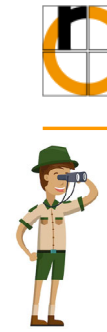
- We read here: *Every person is born in exactly one location and every location any number of persons are born.*

Running example: A Zoo

- Since we are transforming customer requirements into a conceptual data model, we introduce the running example of a zoo and our domain specialist, that will describe the zoo domain for us:

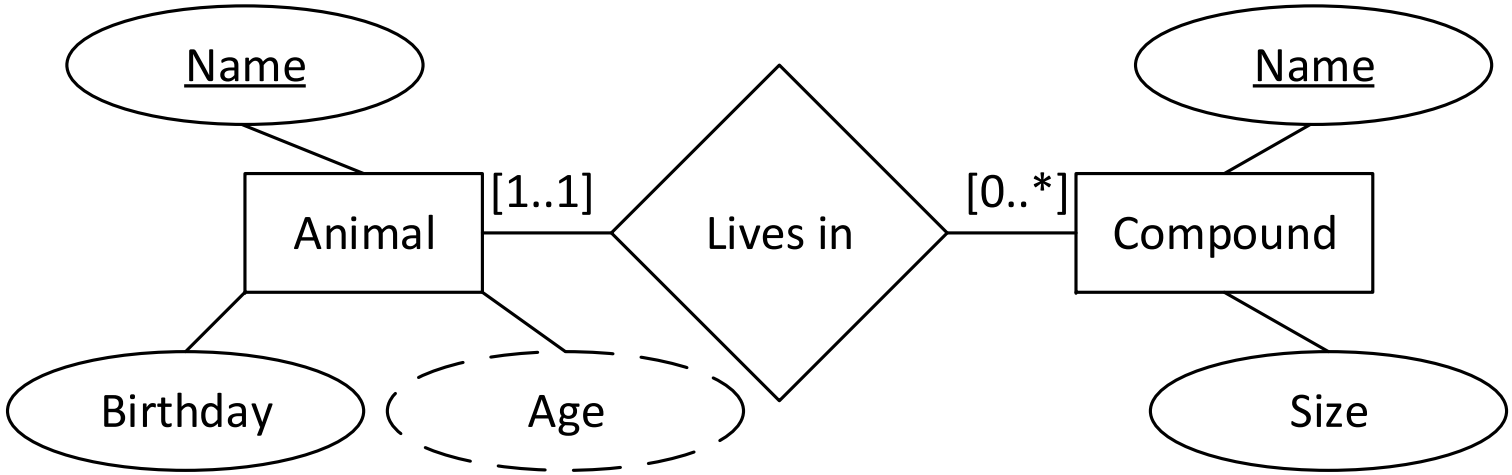


- He has no clue, what a database is, but wants to manage his zoo using IT support and needs to store data.



A simple relationship: one-to-many

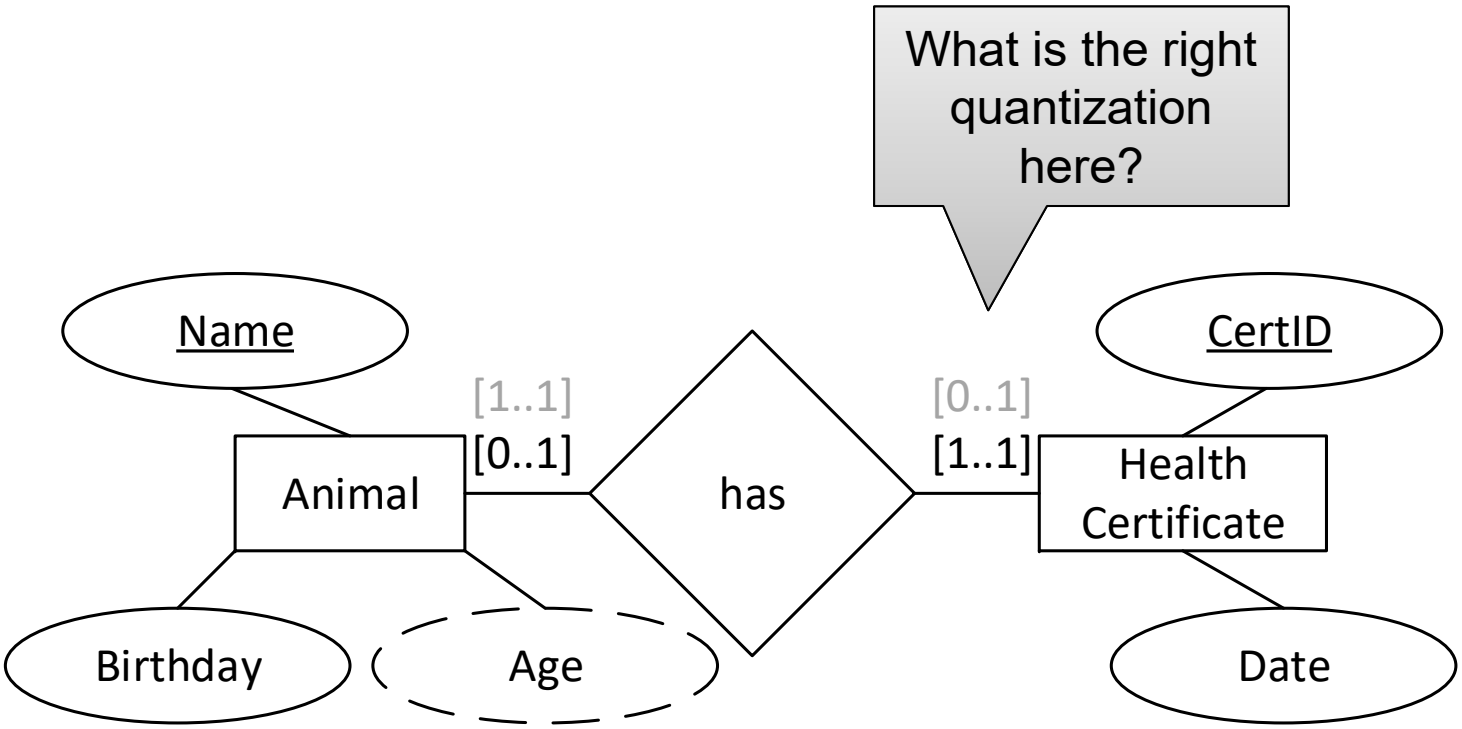
- “Several animals live in each compound. We identify them by a name, and we need to know their age. A compound also has a name and a size in m².”



- An entity (e.g. Animal) represents an element from the real world. It later becomes a relation. Each entity is represented by its attributes. A relationship is represented by a key/foreign key relationship.
- Only primary keys are marked by underlining.

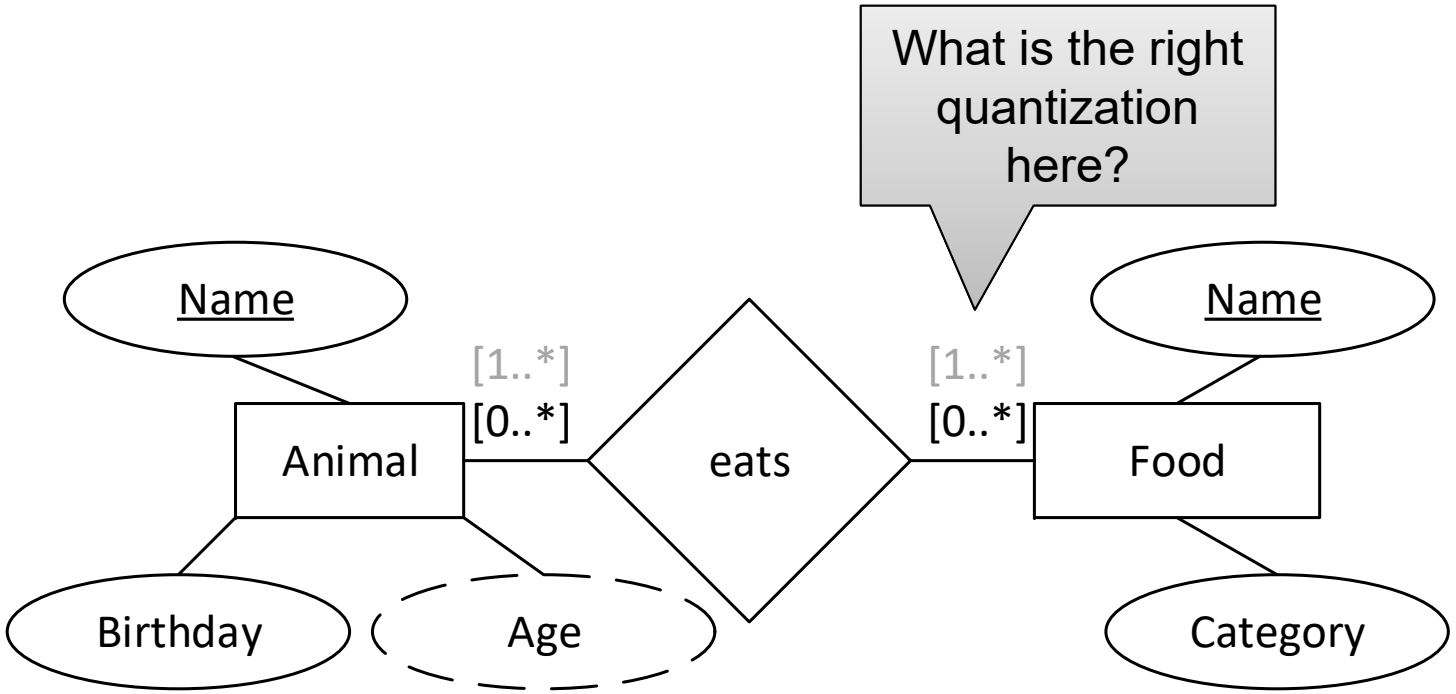
A simple relationship: one-to-one

- “Each animal receives its own health certificate from the vet. That certificate has a date of creation and a certID.”



A simple relationship: many-to-many

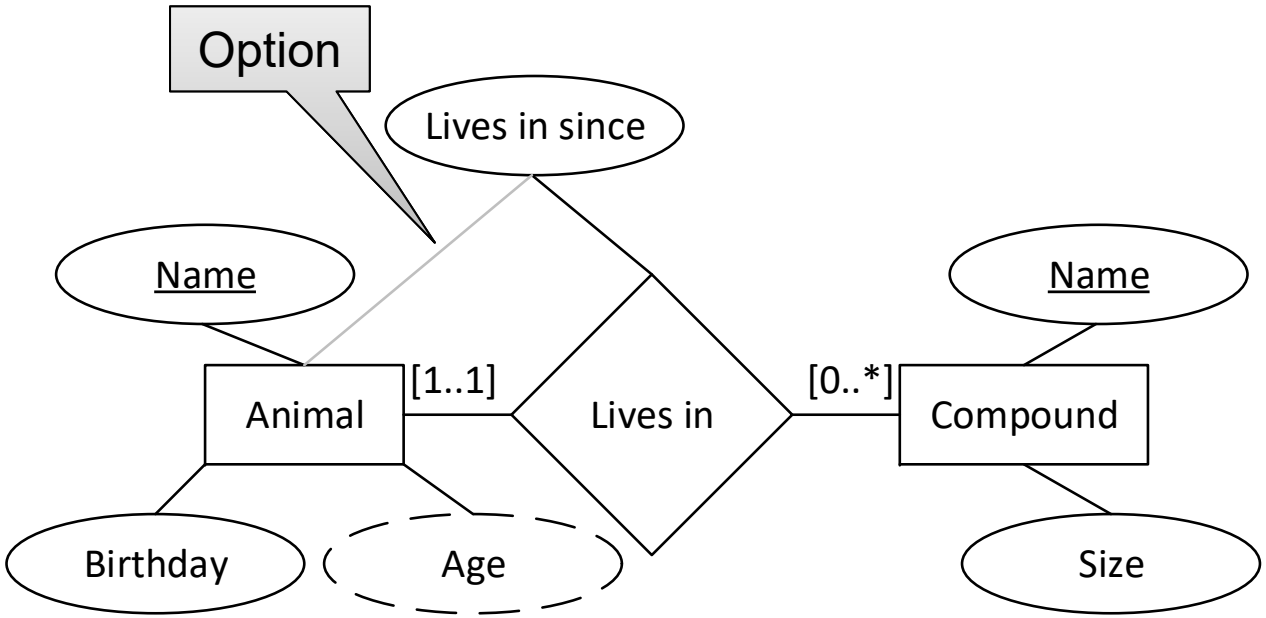
- “We have several types of food for the animals, whereby one animal likes different types of food, and one type of food is good for more than one animal. Each food has a unique name and a health category from A to E.”



- Can a category be also an entity? How would that look like?

Attributes for one-to-many relationships

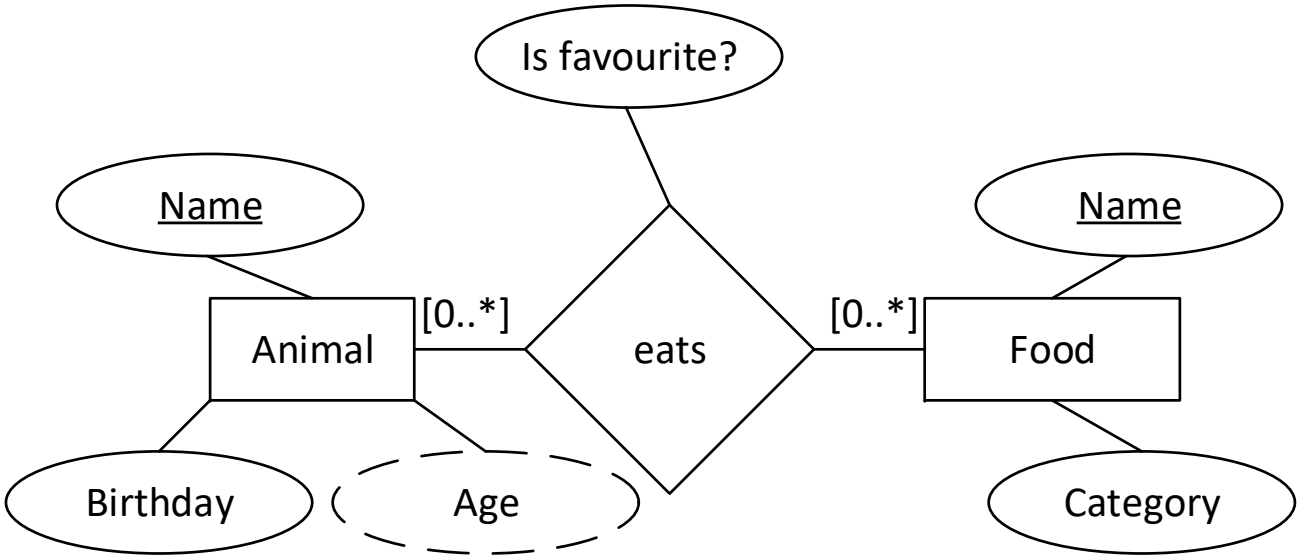
- "… We also want to store, since how long the animals are living in a compound."



- Since each compound can host more than one animal, that cannot be an attribute of a compound. But since every animal lives only in one compound, that could also be an attribute of an animal.

Attributes for many-to-many relationships

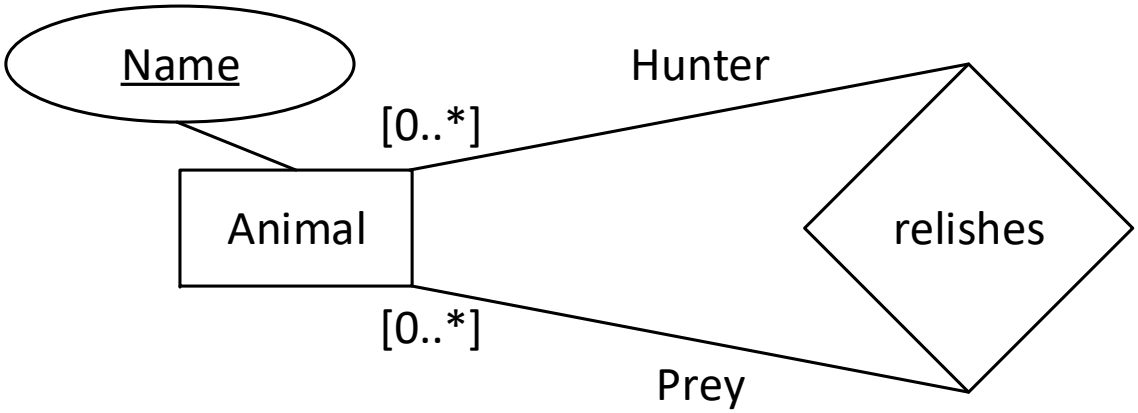
- "… and we want to know, which food is the favourite food of an animal."



- Here, the attribute is favourite? Cannot be stored neither for food nor for an animal, since multiple food entries belong to an animal and a food is consumed by different animals.

Unary relationships

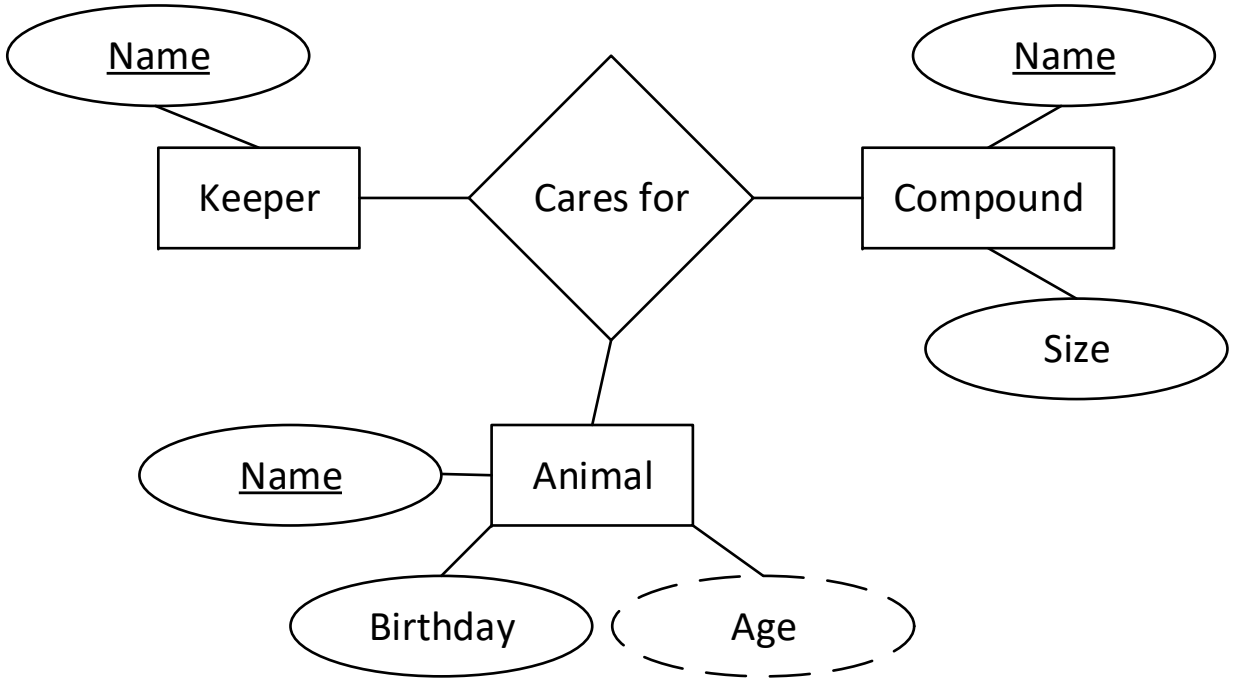
- “Some animals cannot be in the same compound. We need to know which animal will eat another animal.”



- Additionally, roles can be assigned to a relationship. Here can an animal be the hunter of another animal or an animal can be a prey to other animals.

Relationships with more than two entities involved

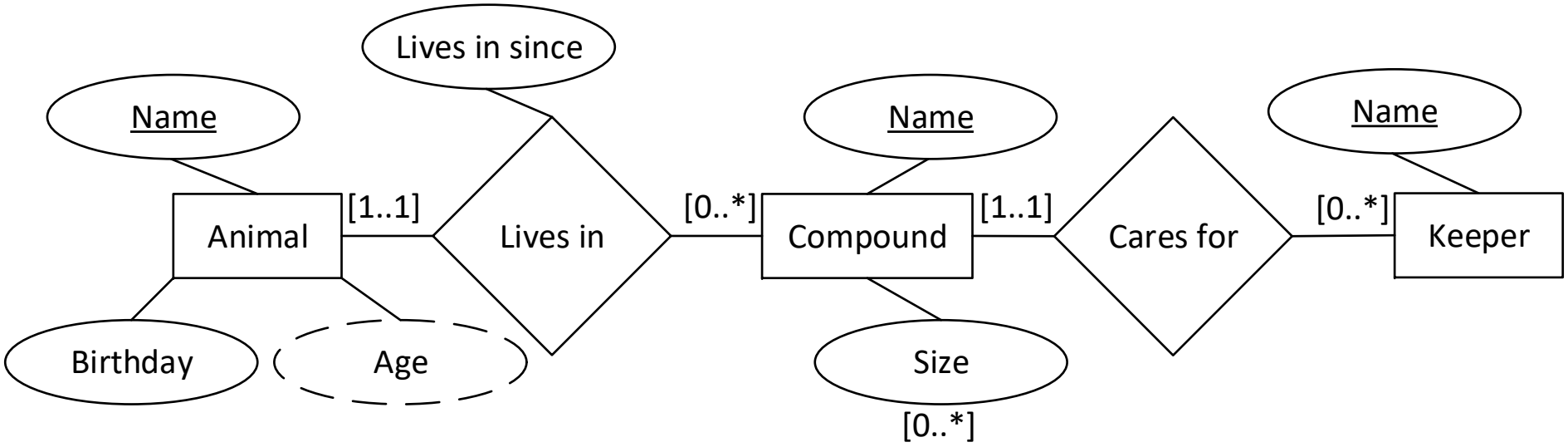
- “Our zookeepers are always responsible for more than one animal. They take care of all animals in one compound.”



- That can be made for modelling but cannot be translated to a relational model. The meaning of a quantization is also unclear and not done here. Binary or unary relationships are preferred.

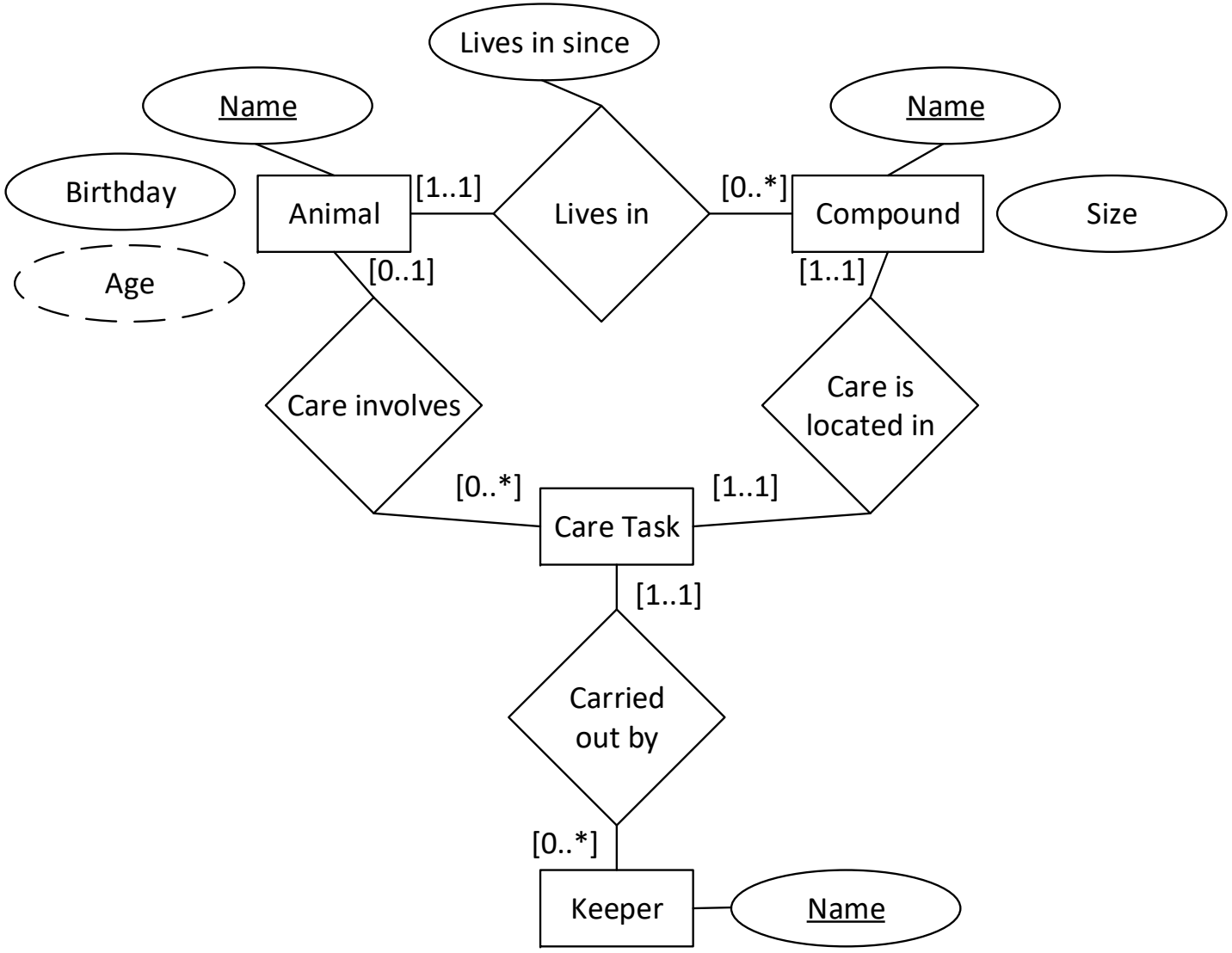
Transforming into binary relationships (1)

- We have some domain-dependent options to use binary relationships instead here, all with some benefits over the others.
- A keeper could be responsible for all animals that belong to a compound:



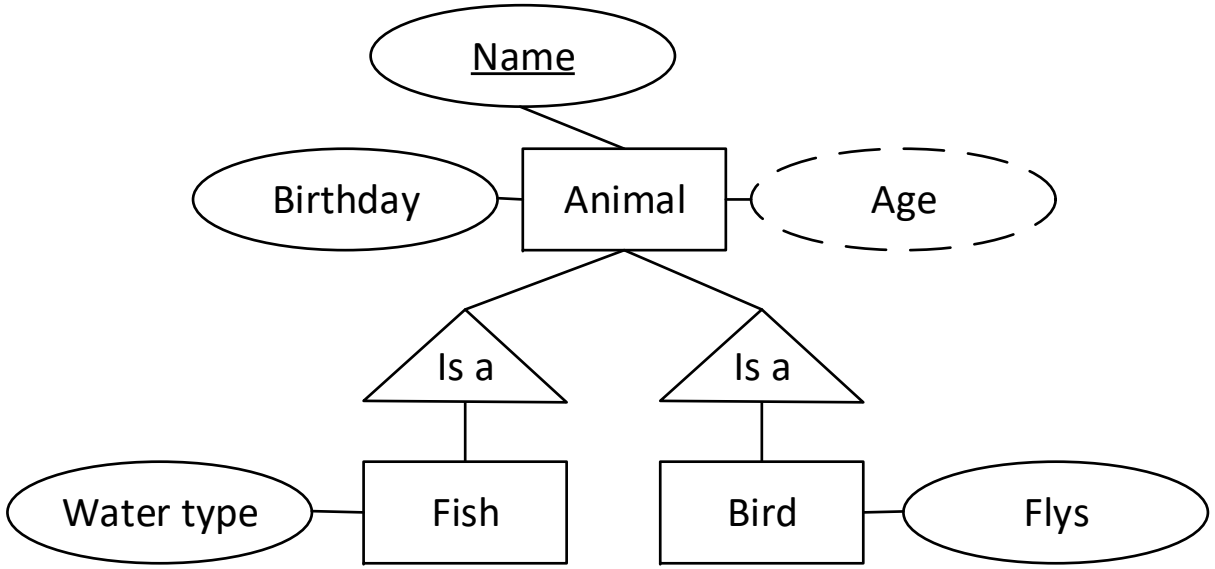
Transforming into binary relationships (2)

- Or there could be a separate entity modelling a care task:



Specialization – Is-a-relationship

- “We also need to make sure to take special care of birds and fish - fish need fresh or salt water, and some birds fly whereas others don’t.”

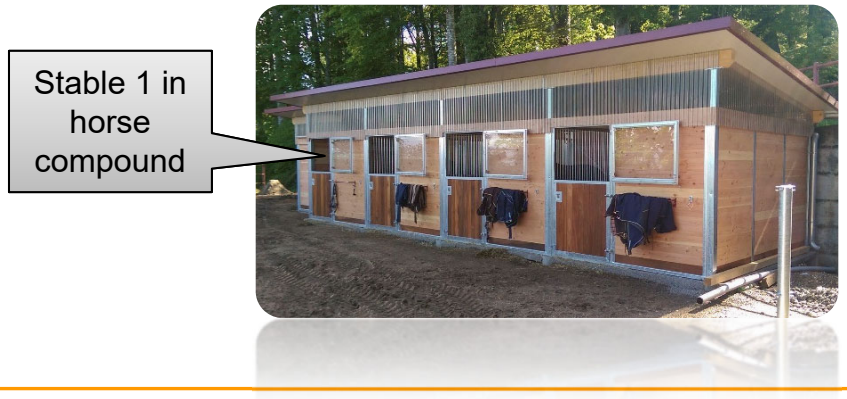


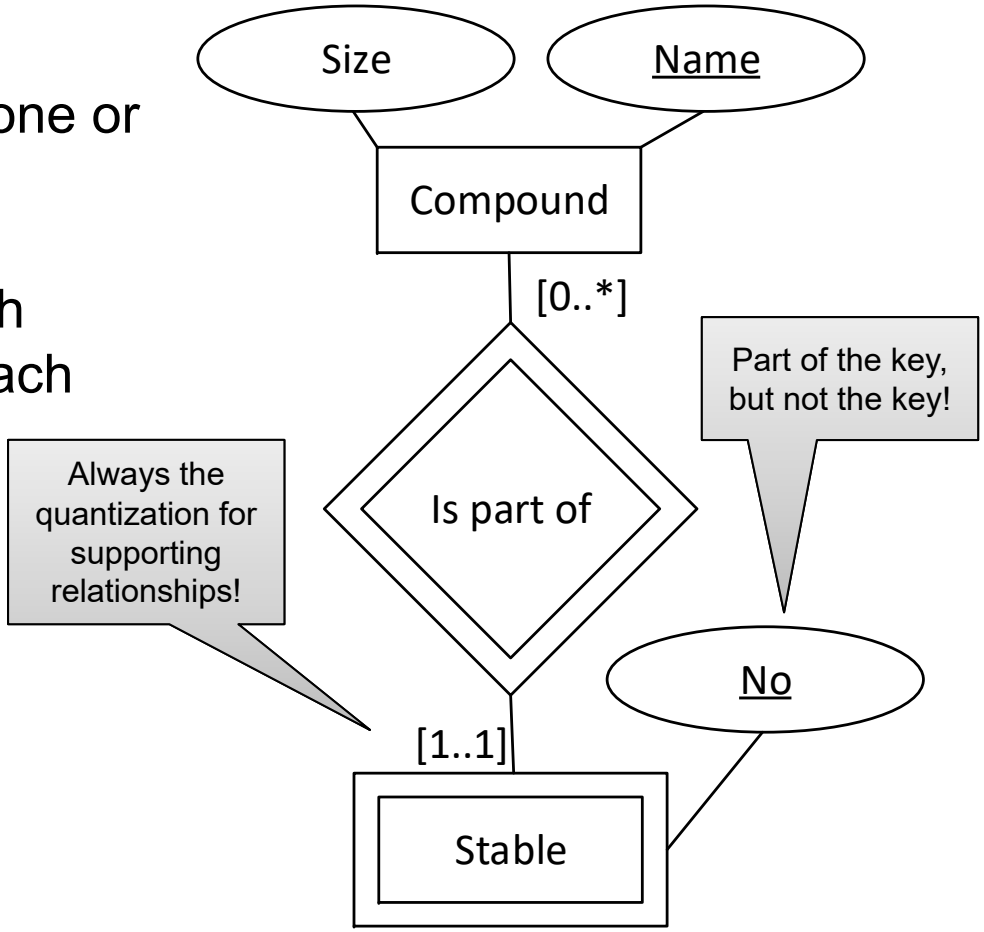
- We will later see, how this can be expressed in a relational model.
- Every root element must contain the key.

Weak entities

- A weak entity is an entity, that has no primary key assigned or just a part of the attributes are part of the primary key. Those entities are marked by a double line.
- A weak entity completes its key using one or more supporting relationships.

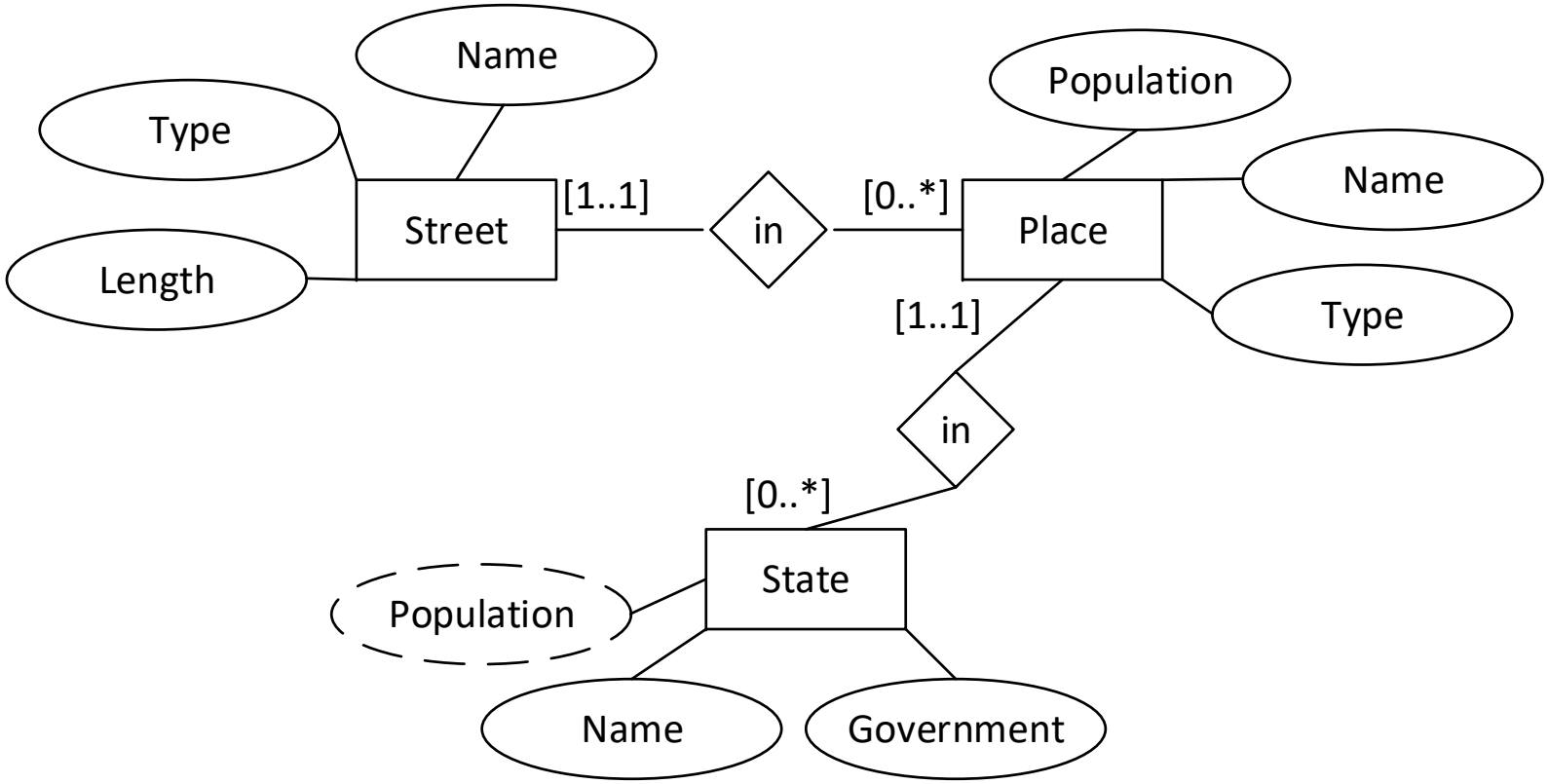
- "A compound has a unique name. Each compound contains several stables. Each stable *within* a compound is uniquely identified by a number."





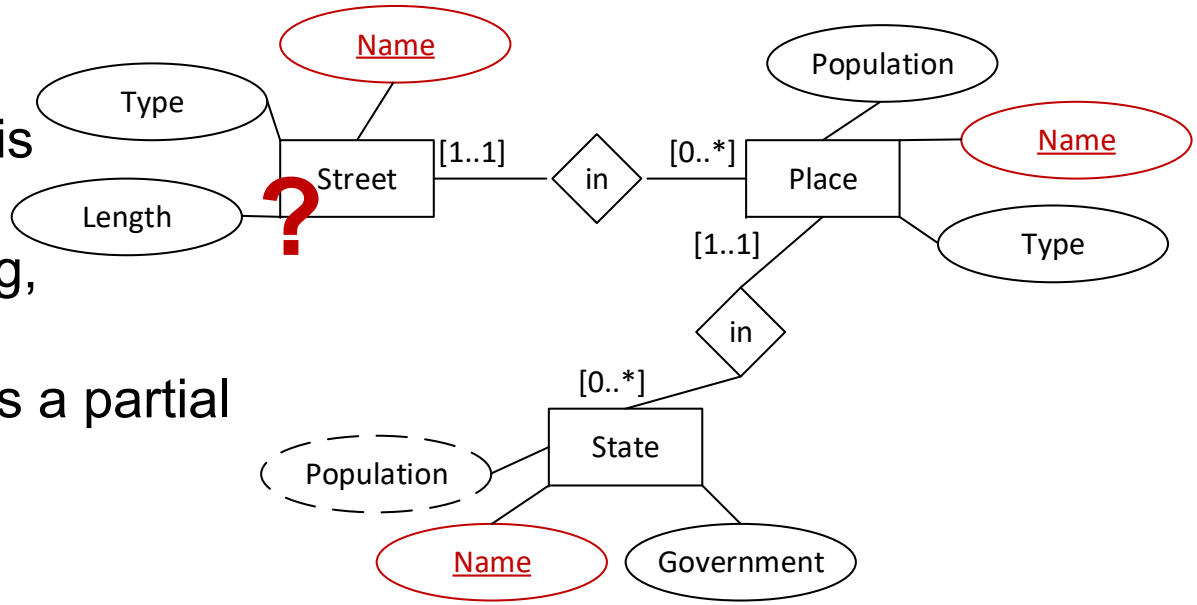
How do we get to weak entities and supporting relationships? Step 1

- Create the ER diagram as usual.



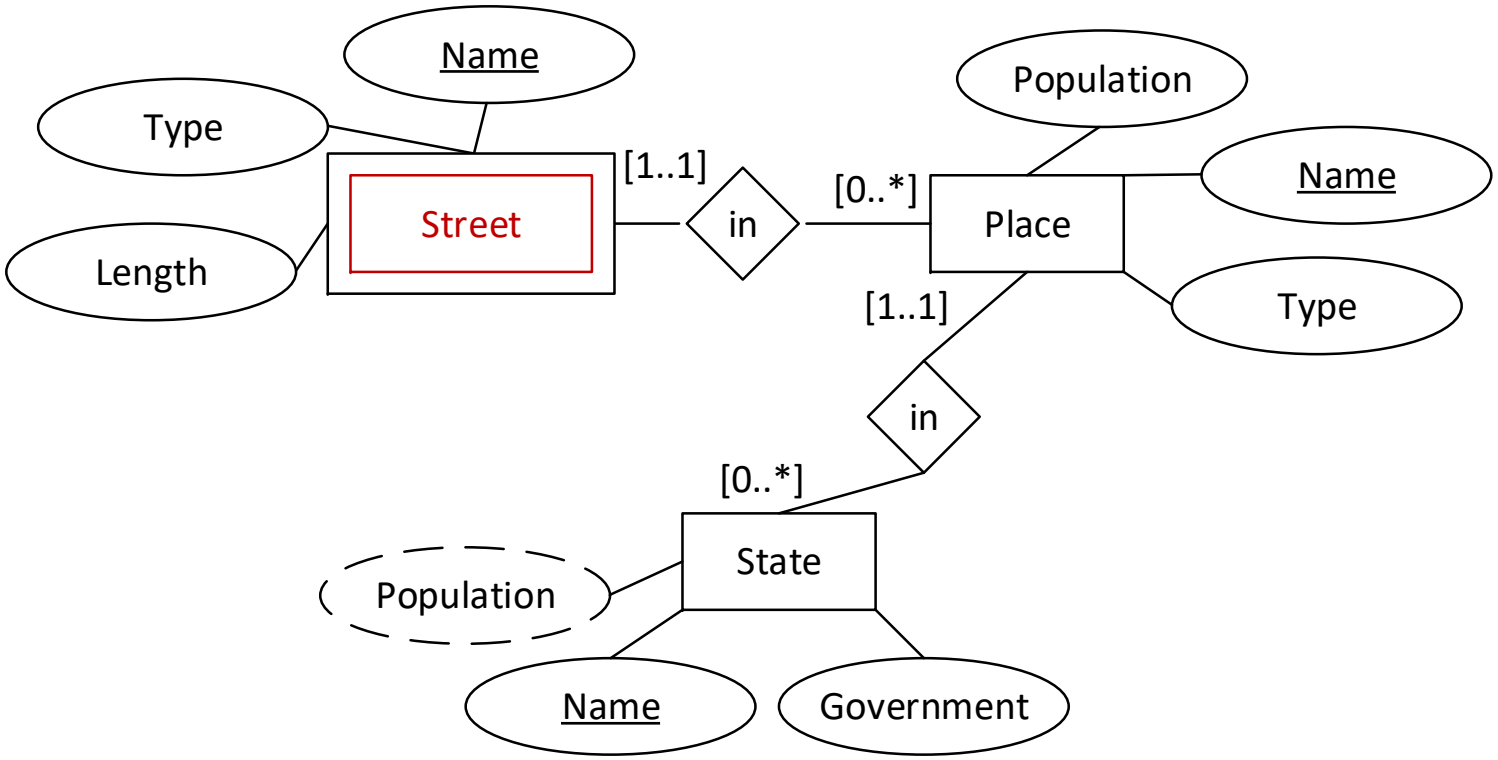
How do we get to weak entities and supporting relationships? Step 2

- Specify the key attributes by underlining them.
- In this example, this works well with state, because the name attribute uniquely identifies a state, so it is a key.
- In the same way, we now initially assume that every place only exists once. Then the name of the place uniquely identifies a place, so it is a key.
- **That doesn't work for street.** Because the name of a street is not unique. There can be a Hindenburgstrasse in Hamburg, but there can also be one in Munich. Nevertheless, Name is a partial key, so underlined.



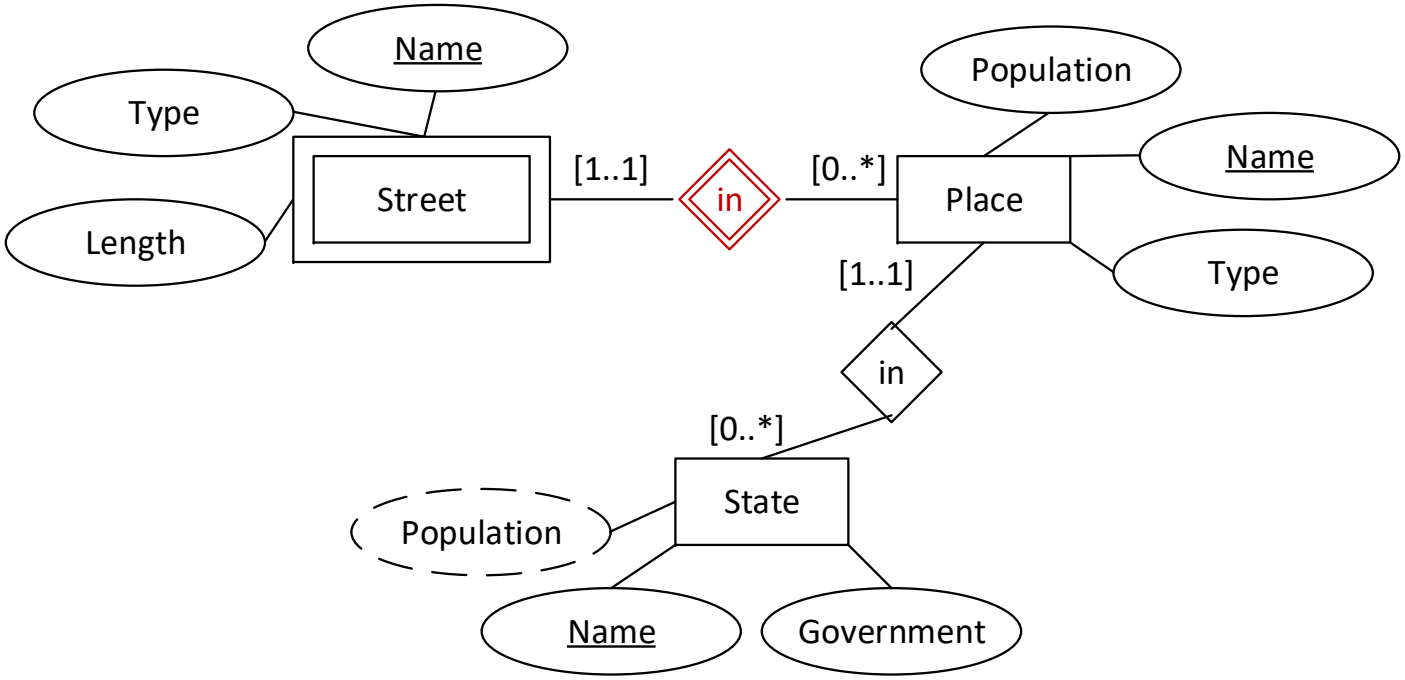
How do we get to weak entities and supporting relationships? Step 3

- Specify the weak entities.
  - All entities that do not have a key yet are weak



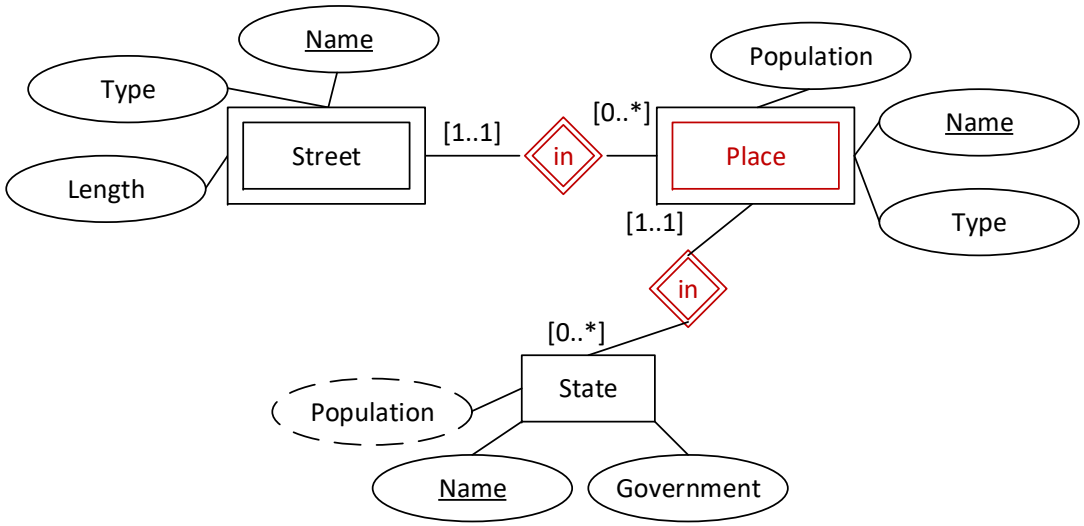
How do we get to weak entities and supporting relationships? Step 4

- Search for keys via supporting relationships
  - In this case, we assume that a place does not have two streets with the same name. As the place name is unique, the combination of street name and place name is also unique. Through the "located in" supporting relationship, we can determine further key attributes which create a key in combination with the name of the street. *Finished.*



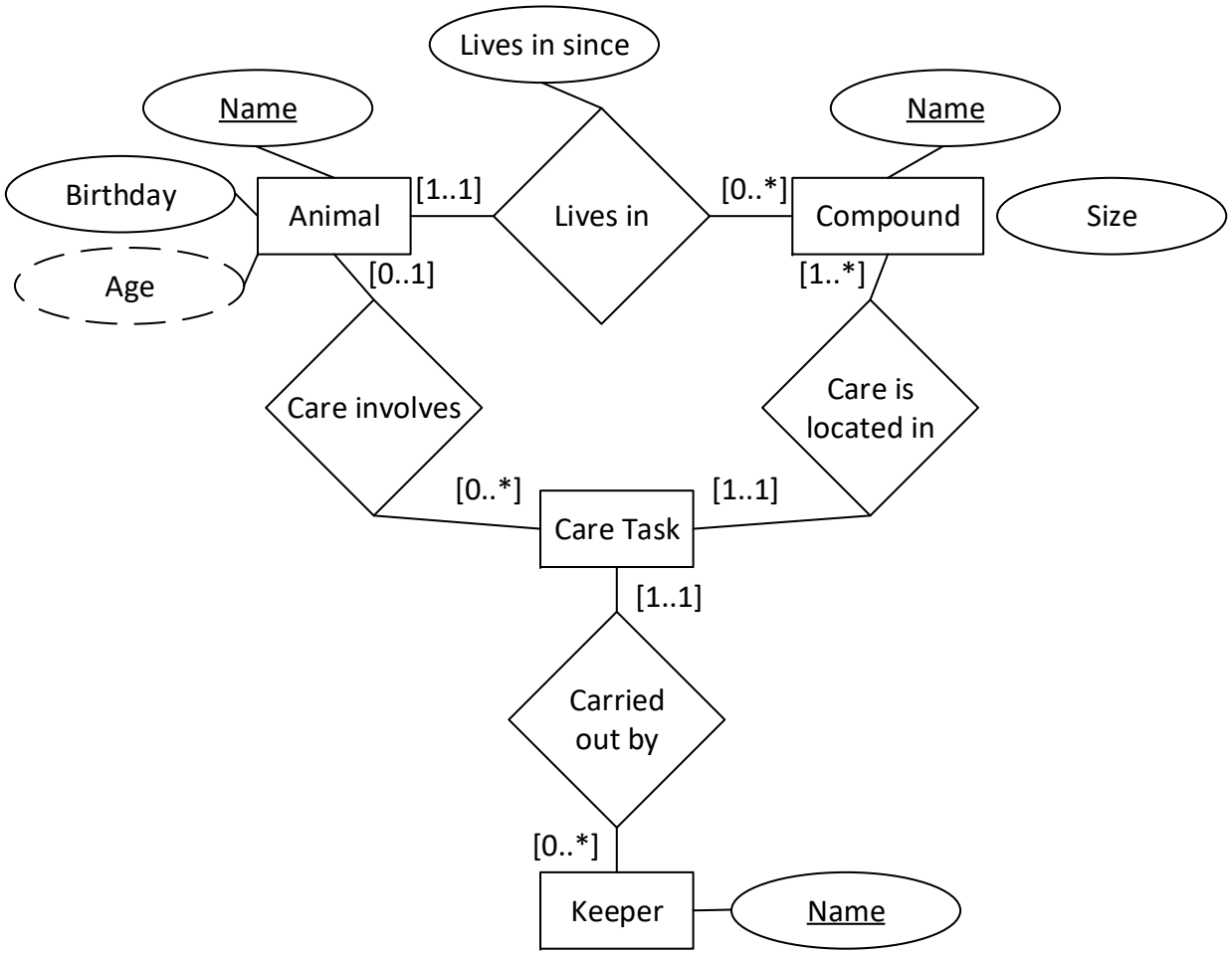
What if...?

- We assume that there may well be two cities with the same name. However, we assume that there are no cities with the same name within one state. Then city name is not a key of place, and place is therefore a weak entity.
- Then "belongs to" is a supporting relationship, and the combination of state name and place name is a key for place.
- In addition, the key of street is the **street name** and all are keys of the supporting relationship, i.e. **place name** and **state name**!



Exercise: What about our Care Task?

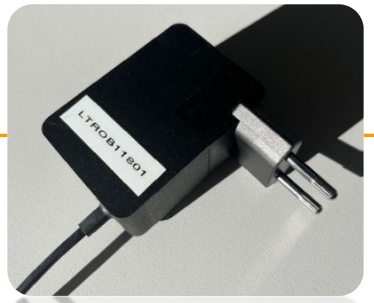
- Care task is weak. It has no key. What are our options here?



Exercise: Device ID

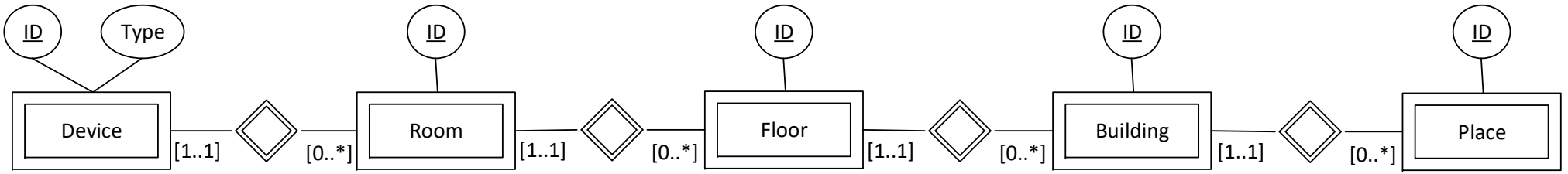
- LT **T**ype
  RO **P**lace
  B **B**uilding
  1 **F**loor
  18 **R**oom
  01 **D**evice

Exercise: Device ID



- LT **T**ype
  RO **P**lace
  B **B**uilding
  1 **F**loor
  18 **R**oom
  01 **D**evice

$$P, B, F, R, D \quad \rightarrow \quad T$$



A modelling guideline for good ER diagrams

- Keep it simple, choose meaningful names, map the reality.
- Entities with no attributes can be hint of an unnecessary entity.
- Cycles in relationships can be a hint of redundant modelling.
  (Zoo example?)

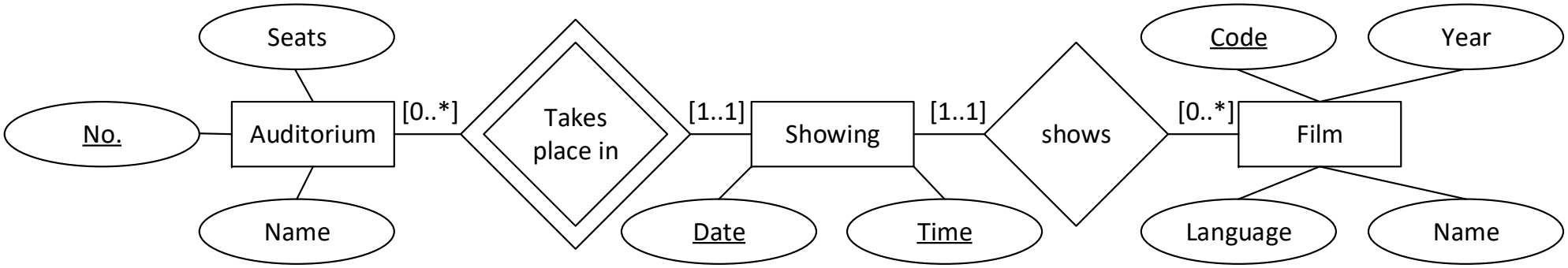
1. Start by modelling an entity with its complete set of attributes (**~2P**)
2. Model relationships and add quantization (**~3P**)
3. Underline keys (**~1P**)
4. No key or just a part?
   Mark it as weak and identify supporting relationships (**~4P**)

Exercise: the multiplex cinema

Let’s look at the operator of a multiplex cinema. The multiplex cinema has several auditoriums. Each auditorium has a name, a unique number and contains a number of seats. Each film has a name, was produced in a certain year in a certain language and has a unique code. A film showing takes place on a particular day at a particular time in an auditorium. Obviously only one film can be shown in an auditorium at any one time.

Exercise: the multiplex cinema

Let's look at the operator of a multiplex cinema. The multiplex cinema has several auditoriums. Each auditorium has a name, a unique number and contains a number of seats. Each film has a name, was produced in a certain year in a certain language and has a unique code. A film showing takes place on a particular day at a particular time in an auditorium. Obviously only one film can be shown in an auditorium at any one time.



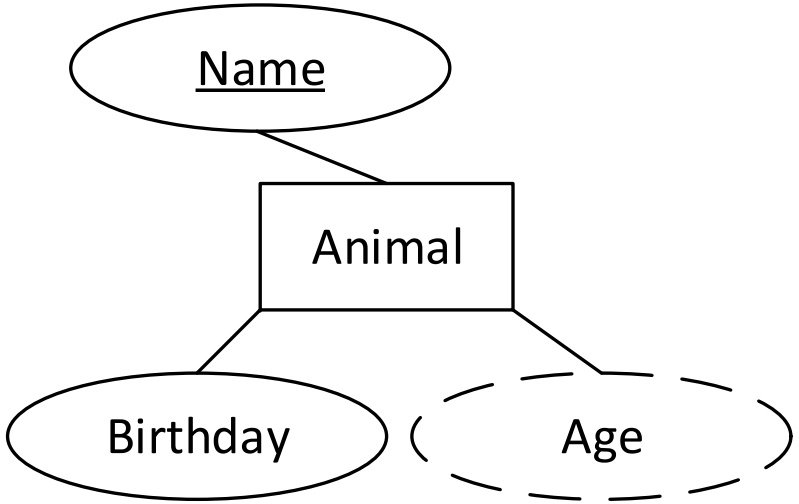
ER mapping

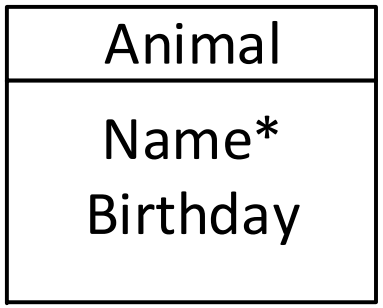
- First partial step of the logical database design
- Mapping of ER model to relational model
- In principle, it's simple:
  - Every entity type becomes a relation
  - Every relationship becomes a relation
- But
  - Merging of relations
  - Weak entity types
  - is-a relationships

Mapping of (simple) entity types

- Every entity type becomes a relation
  - Attributes: all non-computed attributes of the entity type
  - Key: key of the entity type
  - Foreign keys: none
- Example:

ER



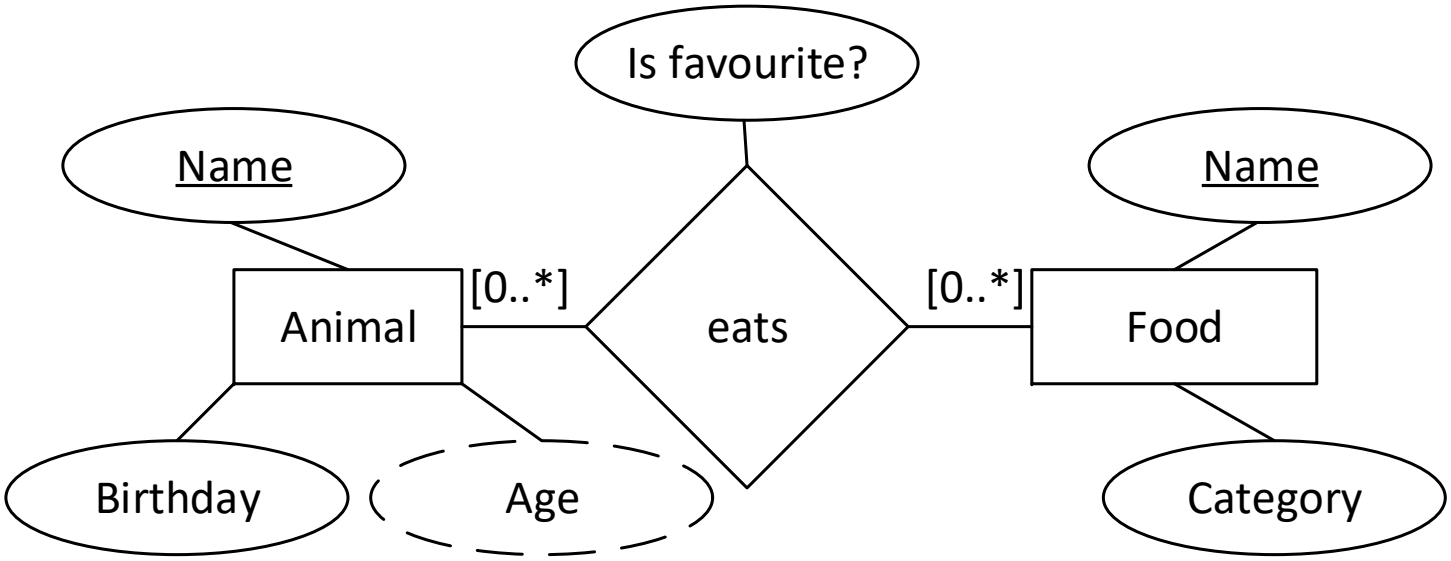


Crowfoot logical design

```
create table Animals(
Name varchar(100) primary key,
Birthday date
);
```

Logical implementation

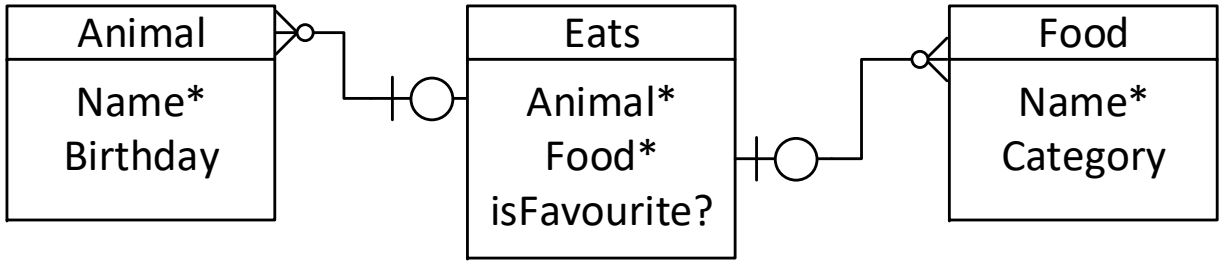
Mapping of many-to-many relationships

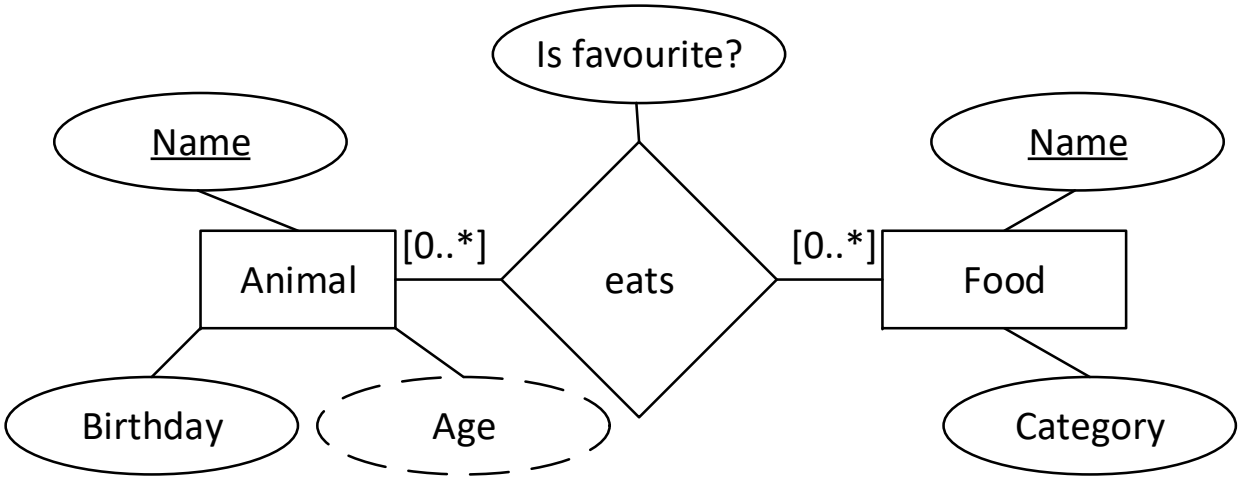


- Quantization cannot be translated clearly using the DDL

```
create table eats(
Animal varchar(100) references Animals(Name),
Food varchar(100) references Food(Name),
Is_favourite bit,
primary key (Animal, Food)
);
```

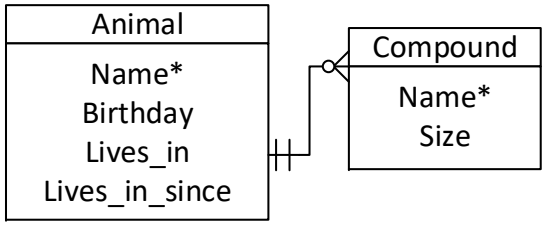
Crowfoods and classic ER Diagram

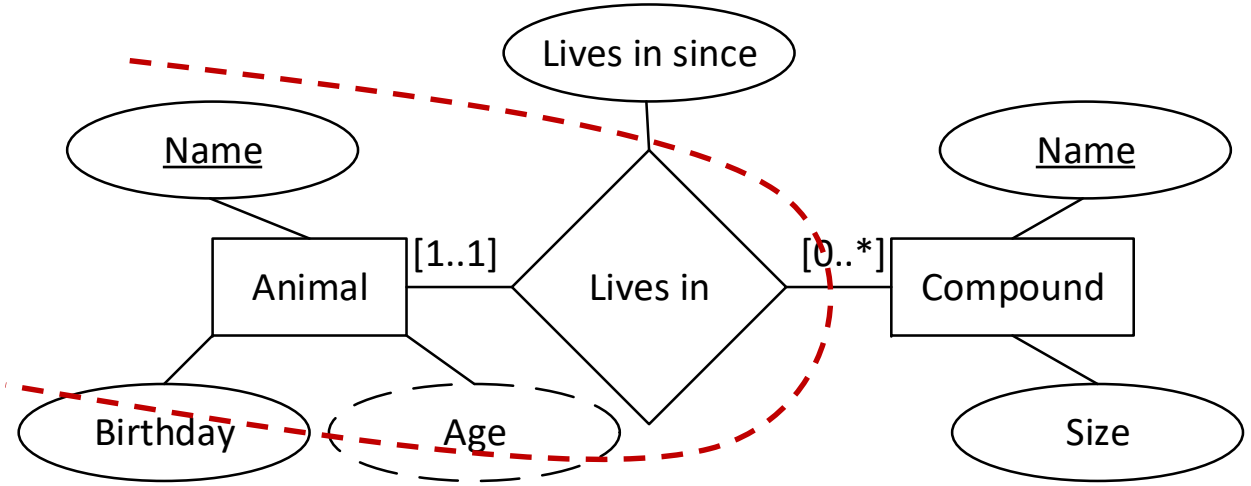




- In the classic ER Diagram, there is a many-to-many relationship, in the Crowffods Digarm, there are many-to-zero/one relations only, because it is closer to the logical implementation.

Mapping of one-to-many relationships

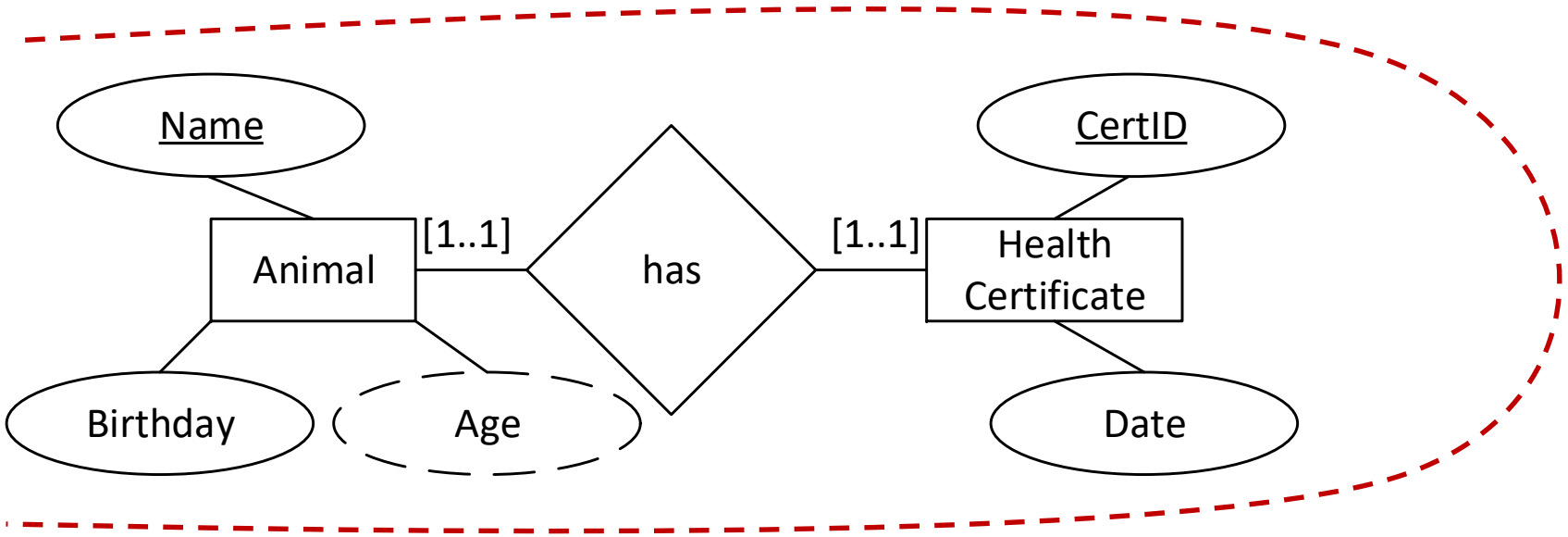





```
create table Animals(
Name varchar(100) primary key,
Birthday date,
Lives_in varchar(100) references Compounds(Name) not null,
Lives_in_since date
);
```

- Relationship is merged to the 1-side, including attributes.
  Quantization can be made using `null` nor `not null` respectively.

Mapping of one-to-one relationships (1)



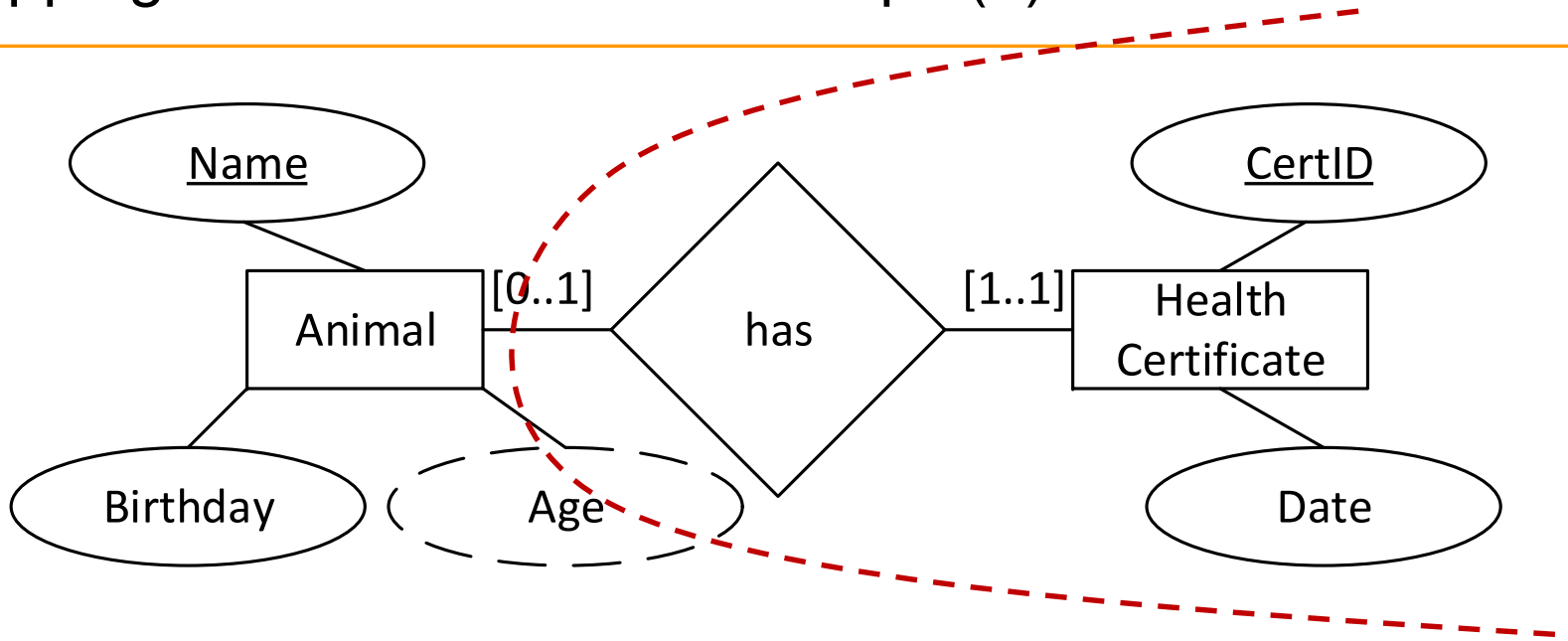
- In this 1-1-case, both entities and the relationship can be merged.

```
create table Animals(
Name varchar(100) primary key,
Birthday date,
Cert_ID int unique,
Cert_date date
);
```

or

```
create table HealthCertificate(
ID int primary key,
date date,
Animal_Name varchar(100) unique,
Animal_Birthday date
)
```

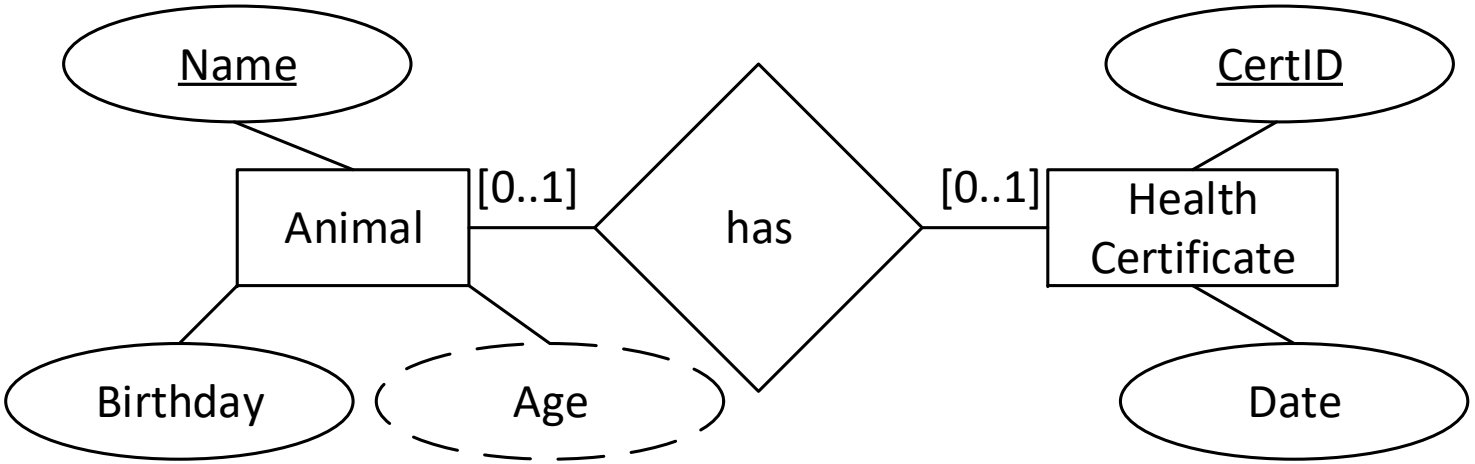
Mapping of one-to-one relationships (2)



- In the 0-1-case, the relationship is merged to the 1-side.

```
create table HealthCertificate(
ID int primary key,
date date,
Animal varchar(100) references Animal(Name) not null
)
```

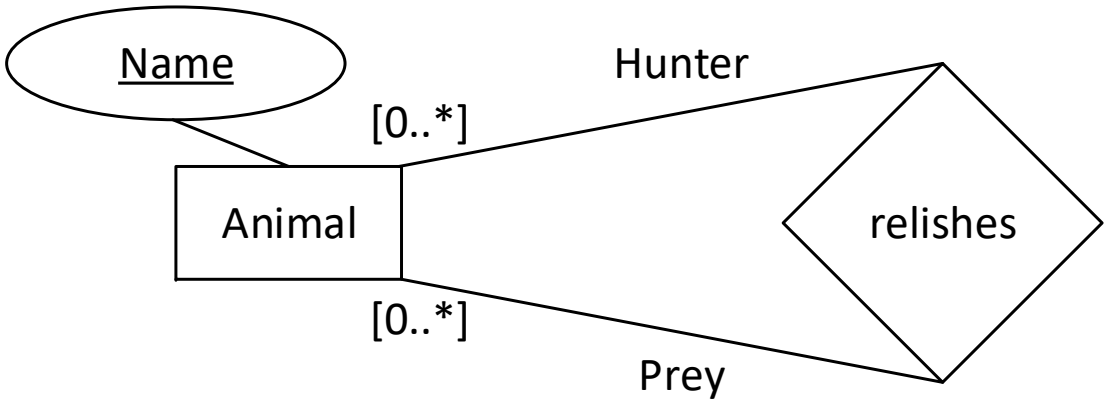
Mapping of one-to-one relationships (3)



- In the 0-0-case, the relationship is not merged.

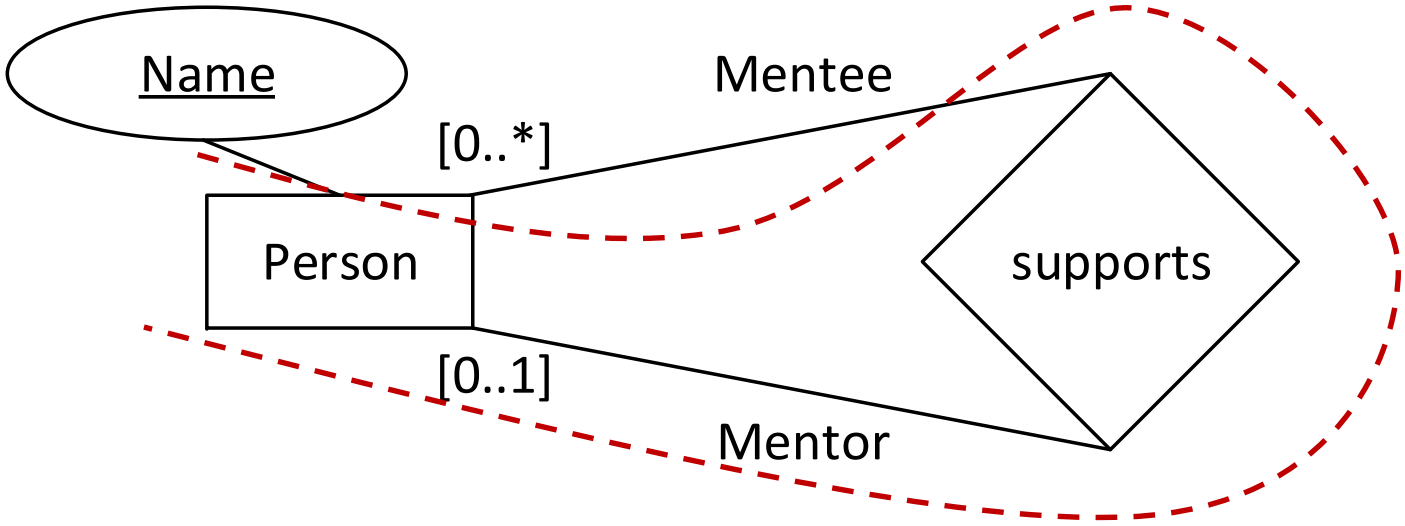
```
create table Animal_has_HealthCertificate(
Animal varchar(100) references Animal(Name),
CertId int references HealthCertificate(ID),
primary key (Animal,CertID)
)
```

Mapping of unary relationships (1)




```
create table Relishes(
Hunter varchar(100) references Animal(Name),
Prey varchar(100) references Animal(Name),
primary key (Hunter,Prey)
)
```

Mapping of unary relationships (2)

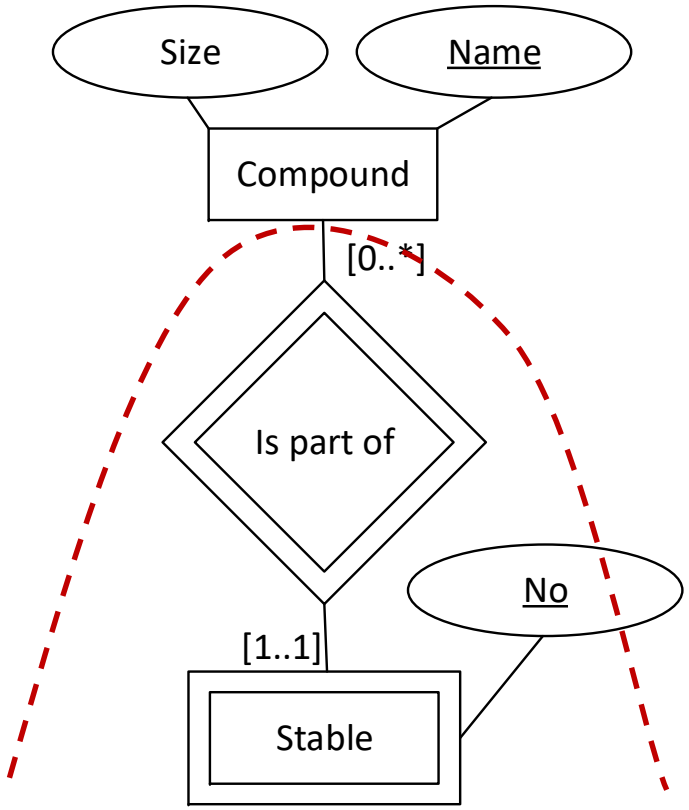



```
create table Person(
Name varchar(100) primary key,
Mentor varchar(100) references Person(Name) null
)
```

- Also merged to the 1-side. `Null, not null` for quantization.

Mapping of weak entities

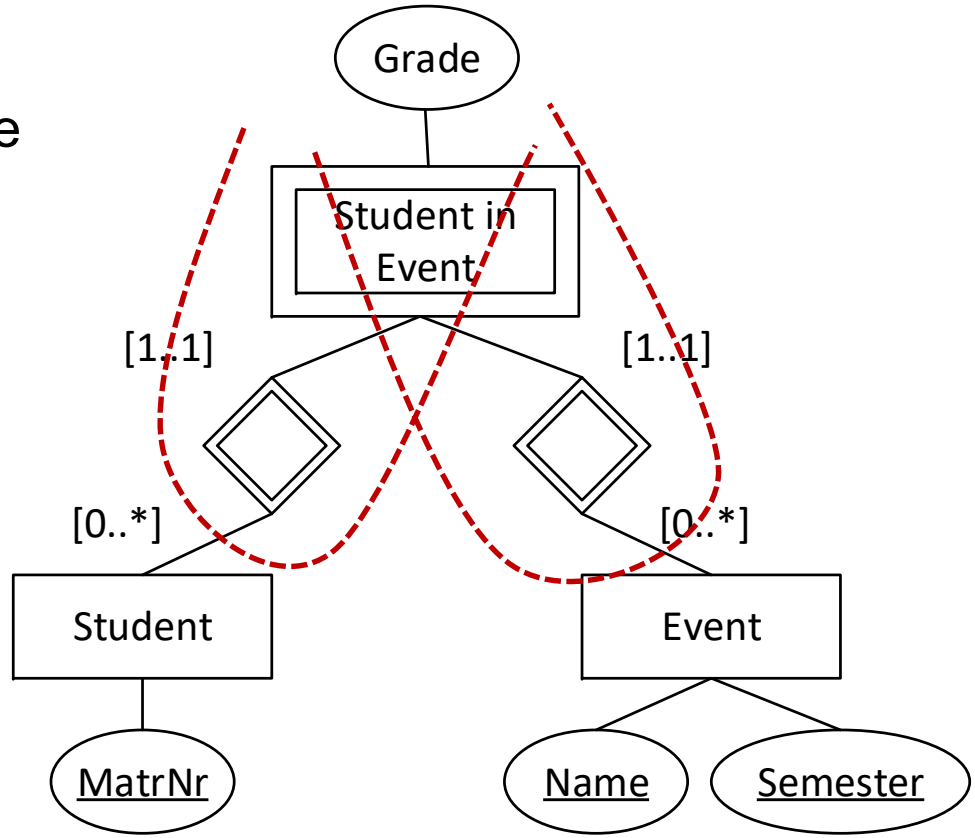
- Weak entities “collect” the key attributes amongst all supporting relationships.They all besome the primary key.
- Supporting relationships always merge to the 1-side.




```
create table Stable(
No int,
Compound_Name varchar (100) references Compound(Name),
primary key (No,Compound_Name)
)
```

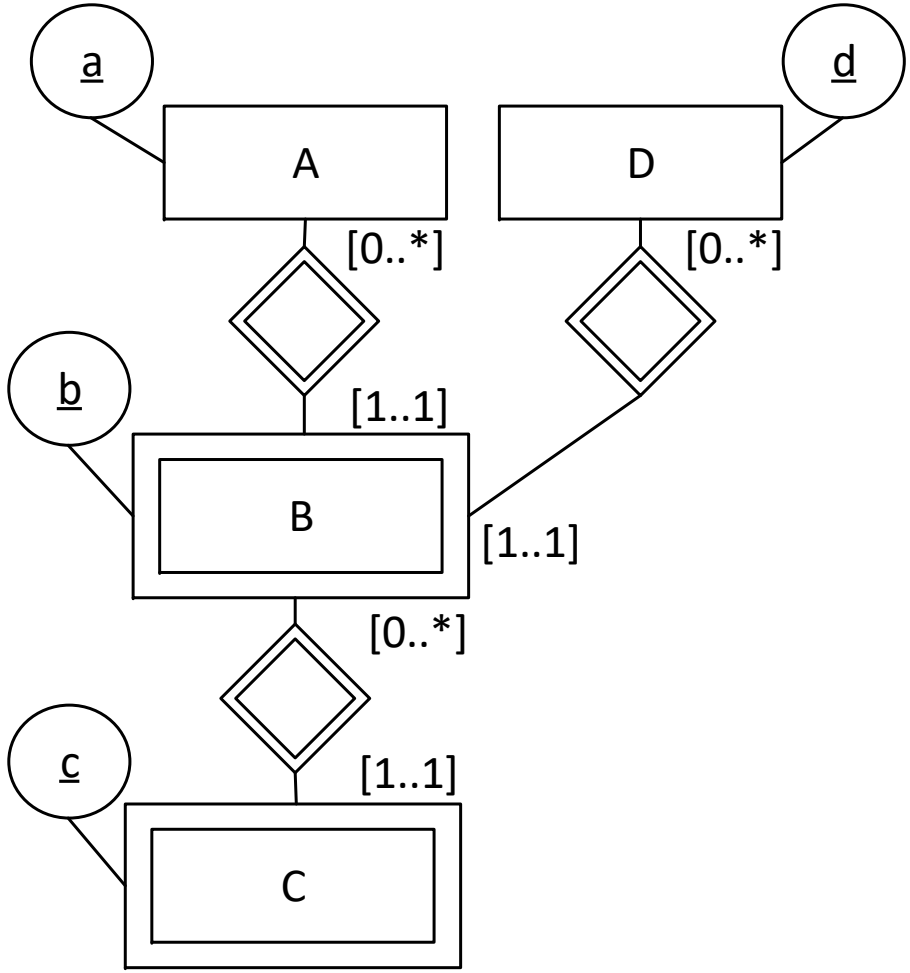
Another Example for mapping weak entities

- This means that `Student_in_Event` has matriculation number, name and semester as its keys (which was also the case in the exercise). Here, the weak entity has no key attribute.



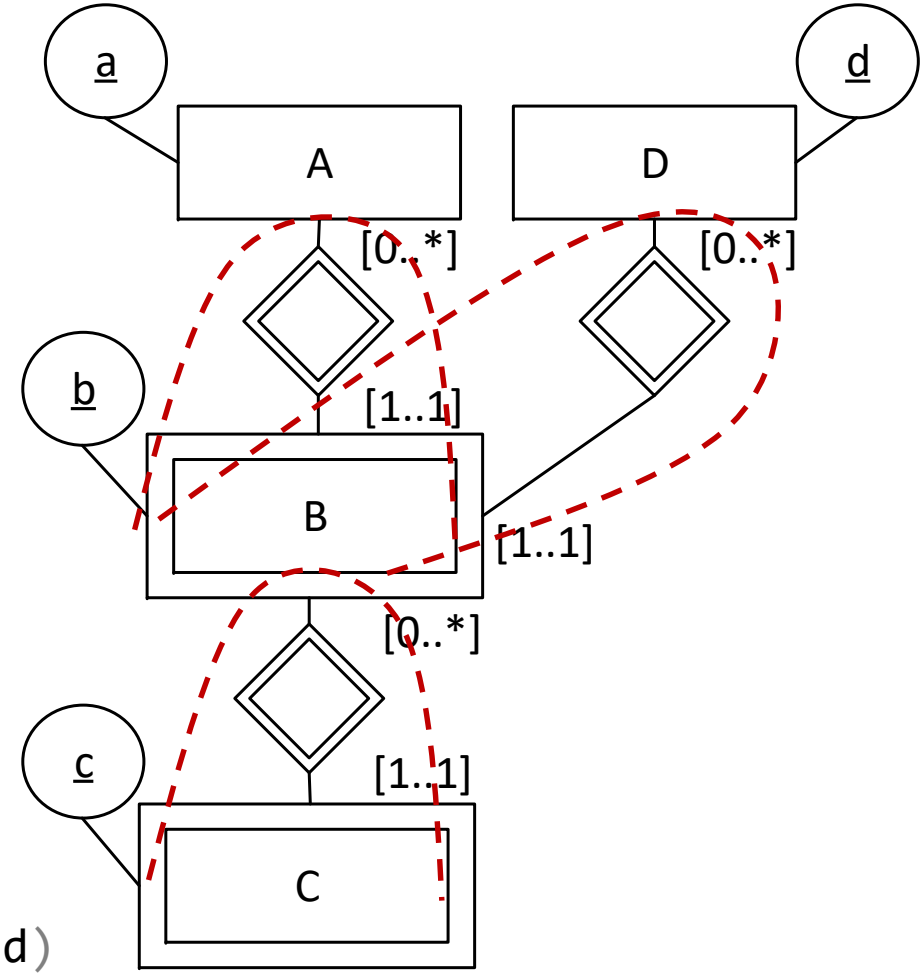

```
create table student_in_event(
grade decimal(2,1),
student int references students(matrnr),
event varchar(100),
semester varchar(4),
foreign key (event,semester) references events(name, semester),
primary key (student, event, semester)
)
```

Exercise: More general example for mapping weak entities



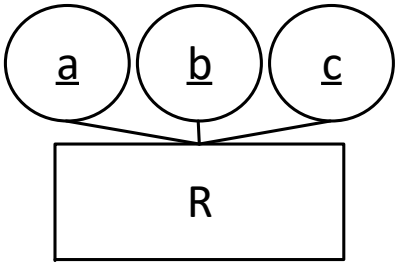
More general example for mapping weak entities

```
create table A(int a primary key);
create table D(int d primary key);
create table B(
  int b,
  int a references A(a),
  int d references D(d),
  primary key (a,b,d)
);
create table C(
  int c,
  int b,
  int a,
  int d,
  primary key (c,b,a,d),
  foreign key (b,a,d) references B(b,a,d)
)
```

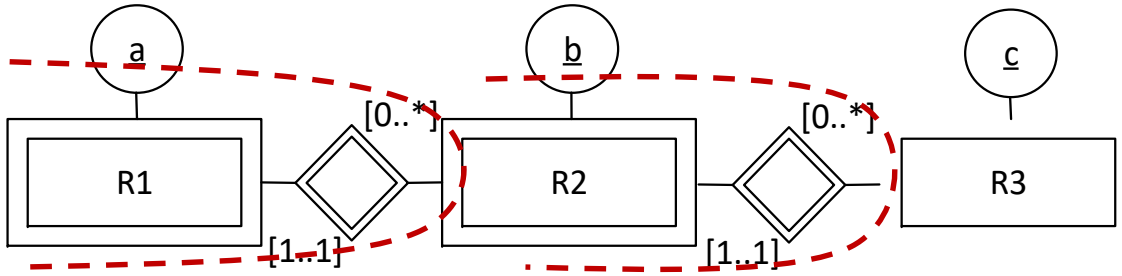


Multiple key attributes can always be distributed via weak entities

- Suppose we have an entity R with three key attributes A,B,C.

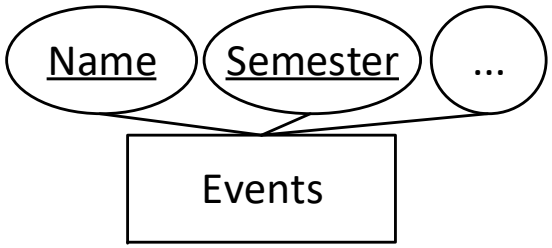


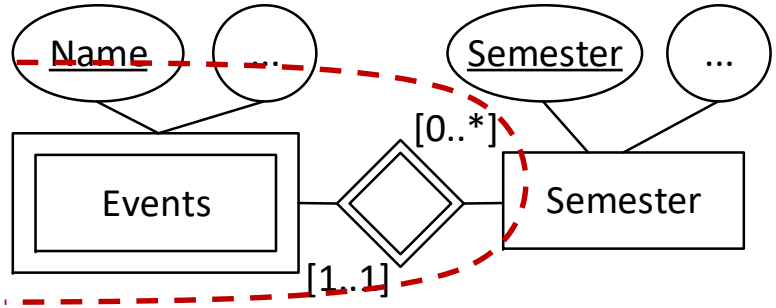
- This entity can also be represented as a weak entity with two supporting relationships



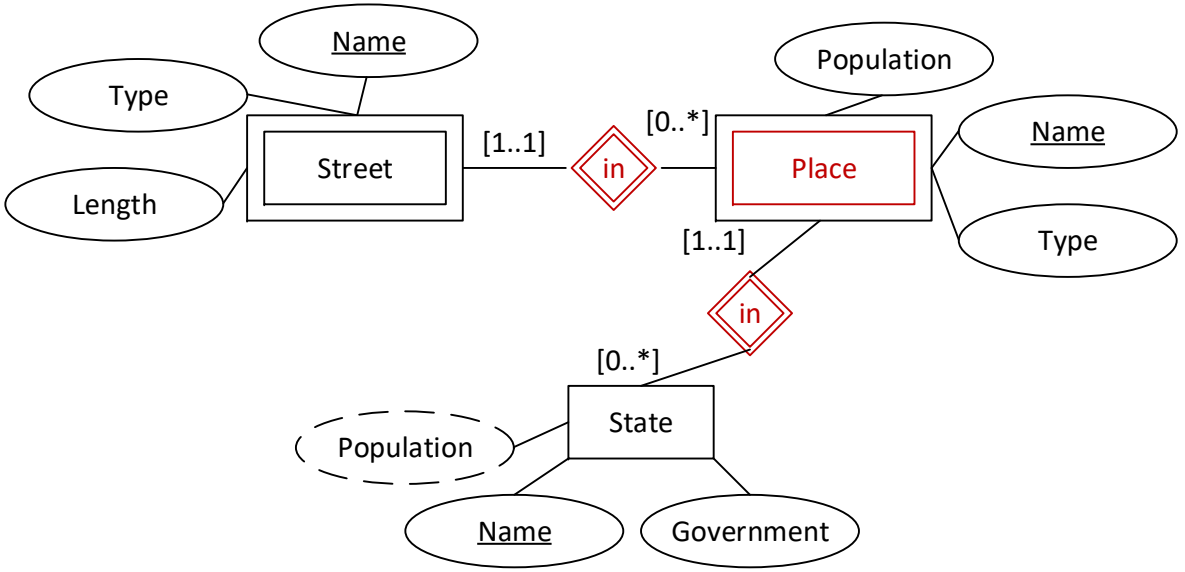
What are the FDs here?

- Example from the exercise. Events had name and semester as their key. We can also model this with a supporting relationship and an additional entity:



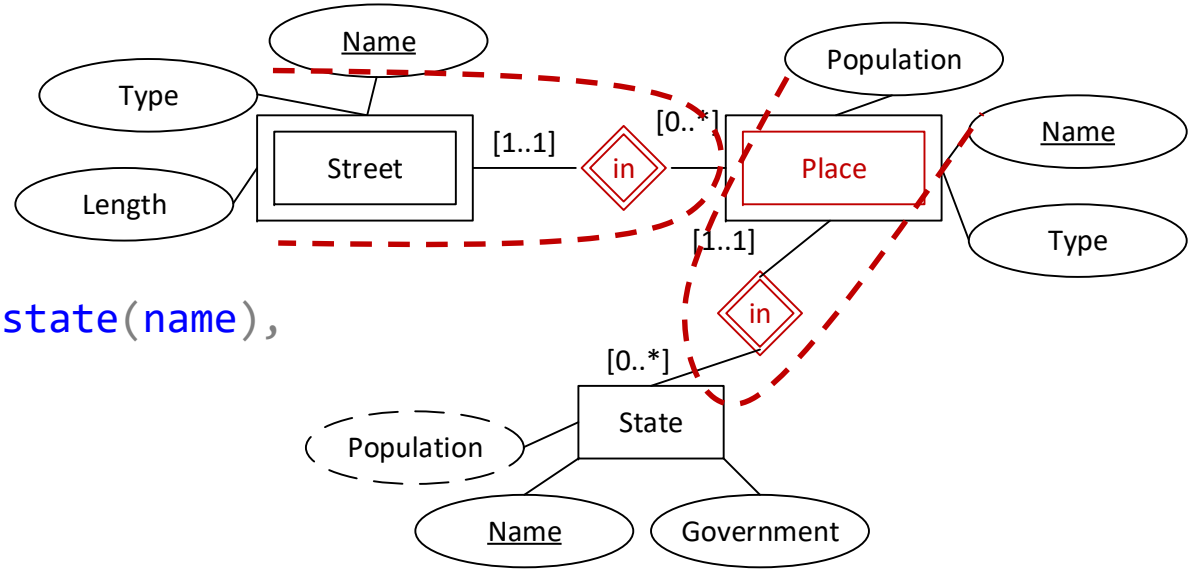


Exercise: Derivation of relations



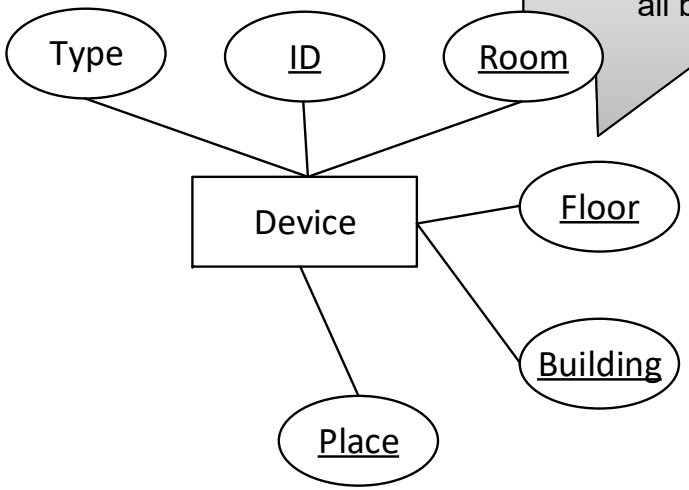
Exercise: Derivation of relations

```
create table state(
name varchar(100) primary key,
government varchar(300)
)
create table places(
name varchar (100),
state varchar(100) references state(name),
population int,
type varchar(100),
primary key (name, state)
)
create table streets(
name varchar(100),
place varchar(100),
state varchar(100),
foreign key (place,state) references
places(name, state),
primary key (name,place,state)
)
```

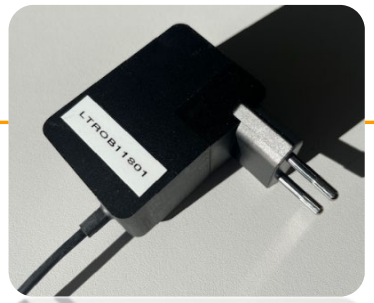


Exercise: Device ID

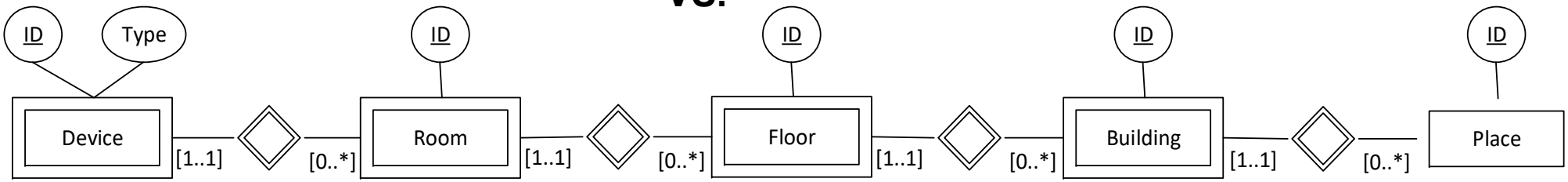
- LT **T**ype
  RO **P**lace
  B **B**uilding
  1 **F**loor
  18 **R**oom
  01 **D**evice



Deletion anomaly? When deleting all devices, I lose all buildings ?!



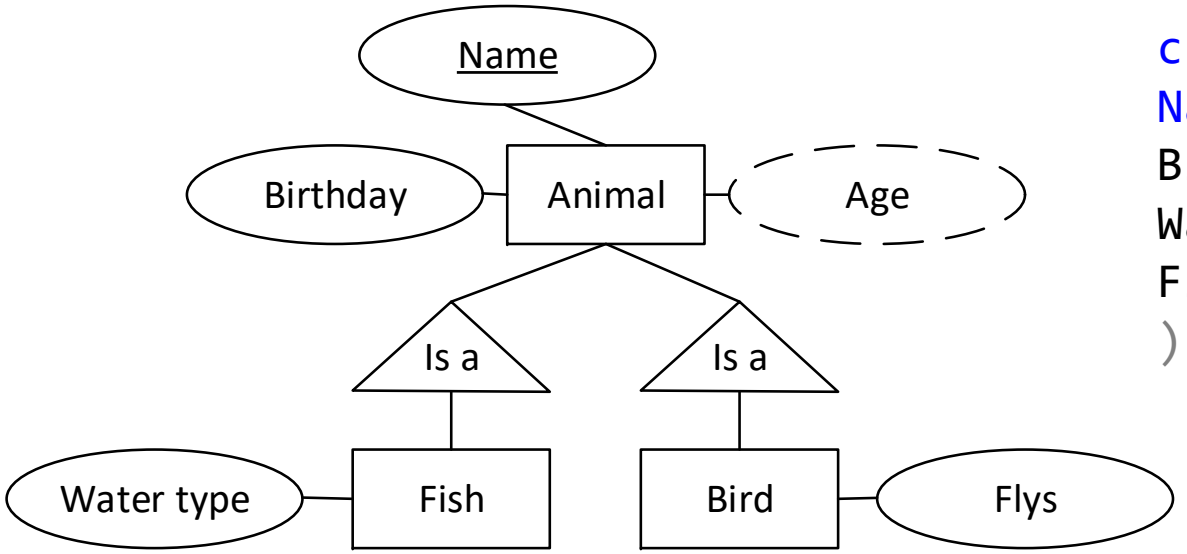
VS.



$$P, B, F, R, D \rightarrow T$$
$$P, B \rightarrow Address$$

Both are equally good, but when the IDs are used elsewhere, e.g. to store an address, they are better stored in separate tables.

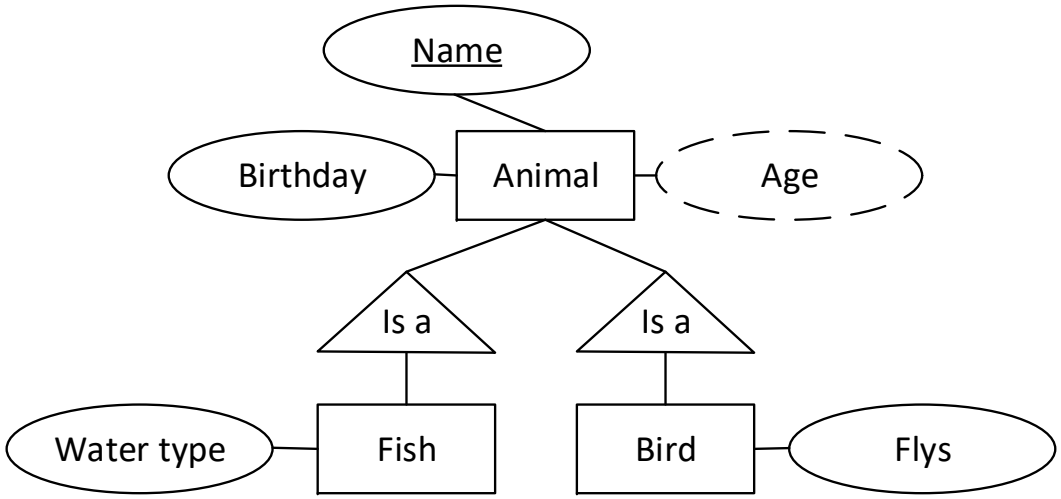
Mapping of is-a relationships – null-value style




```
create table Animals(
Name varchar(100) primary key,
Birthday date,
Water_type varchar(100),
Flys bit
)
```

- Fewer relations, carries lots of `null` values.
- Only one tuple per animal, good for queries, inserts and updates.

Mapping of is-a relationships – ER style

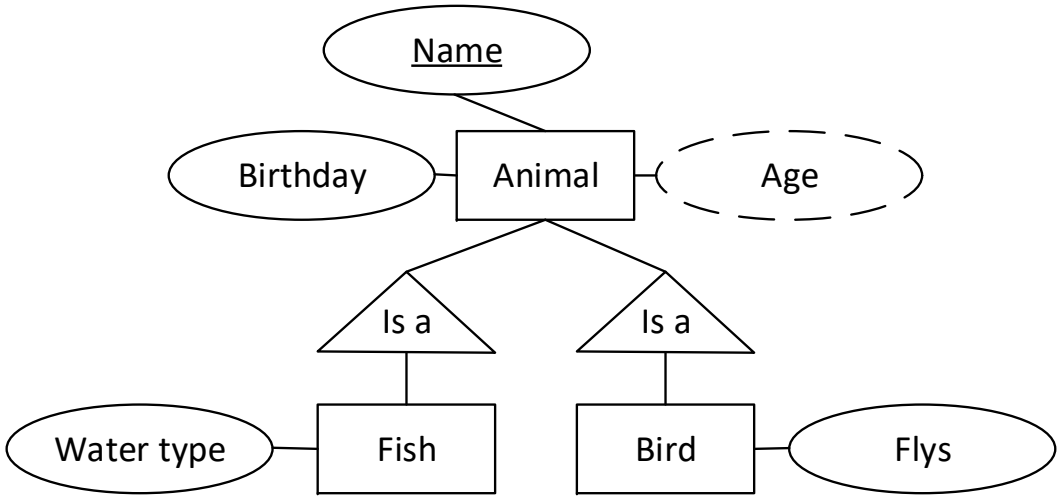



```
create table Animals(
Name varchar(100) primary key,
Birthday date
)

create table Fishes(
Animal varchar(100) references Animals(Name) primary key,
Water_type varchar(100)
)

create table Birds(
Animal varchar(100) references Animals(Name) primary key,
Flys bit
)
```

- More relations, no nulls
- Interactions address more tables.

Mapping of is-a relationships – OO style

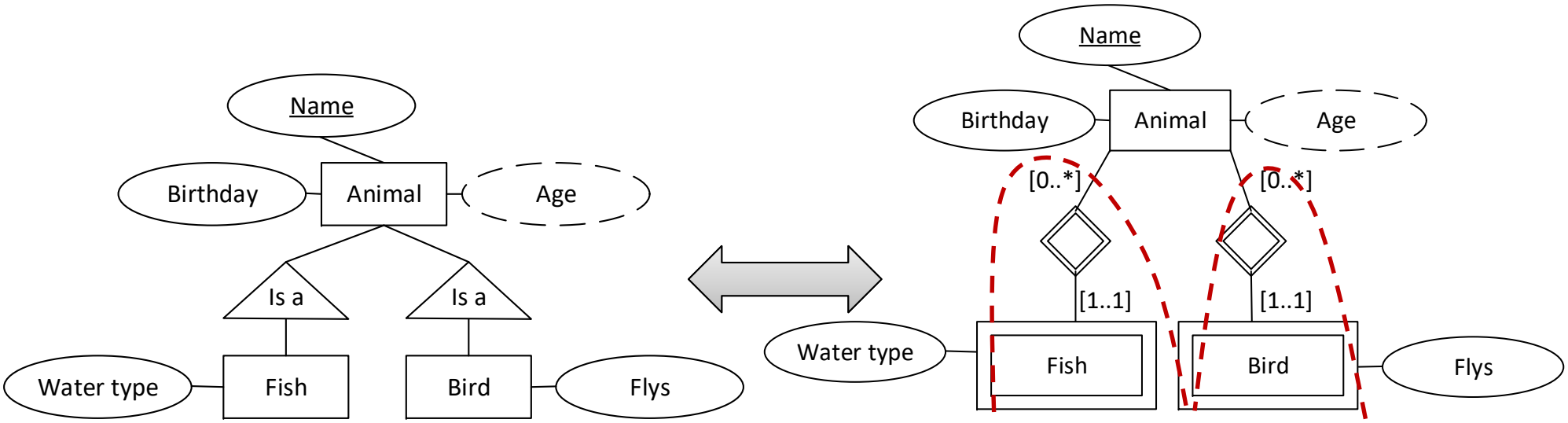
```
create table Animals(
 Name varchar(100) primary key,
 Birthday date
)
create table Fishes(
 Name varchar(100) primary key,
 Birthday date,
 Water_type varchar(100)
)
create table Birds(
 Name varchar(100) primary key,
 Birthday date,
 Flys bit
)
create table BirdFishes(
 Name varchar(100) primary key,
 Birthday date,
 Water_type varchar(100),
 Flys bit
)
```



- Key hard to maintain, but only one table addressed for inserts.
- Many relations

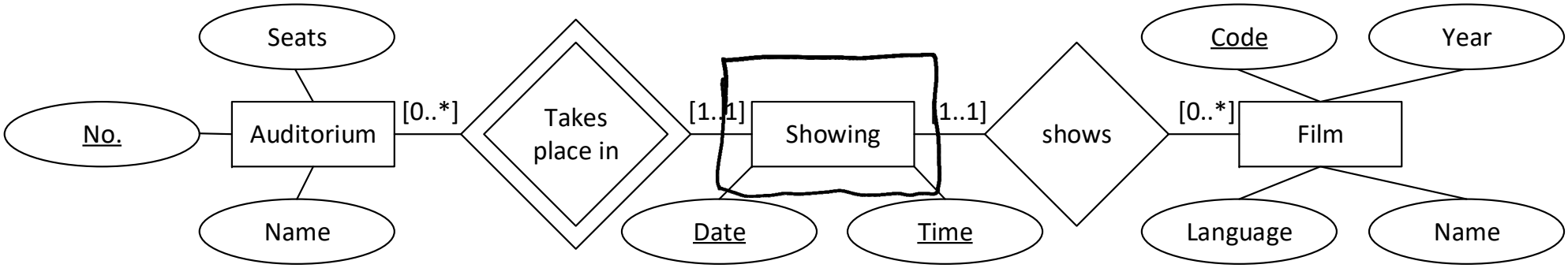
An alternative way to model is-a relationships

- You can also arrive at ER mapping via regular relationships and merging. Example:



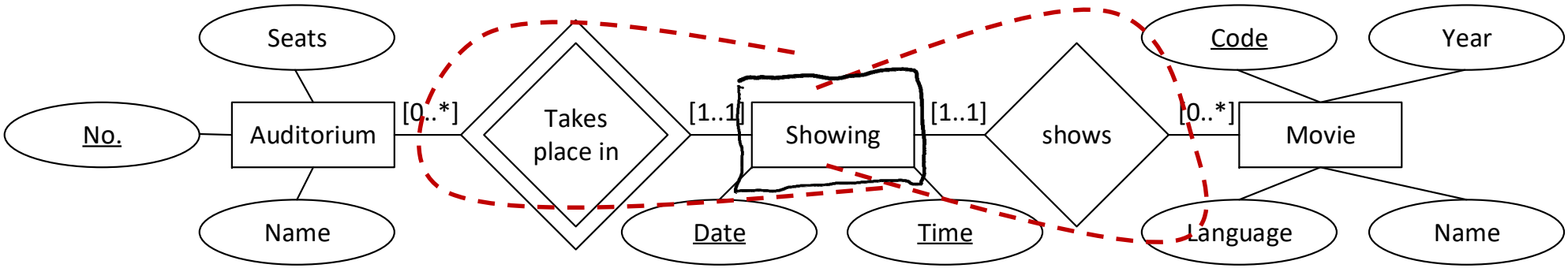
Exercise: the multiplex cinema

Let's look at the operator of a multiplex cinema. The multiplex cinema has several auditoriums. Each auditorium has a name, a unique number and contains a number of seats. Each film has a name, was produced in a certain year in a certain language and has a unique code. A film showing takes place on a particular day at a particular time in an auditorium. Obviously only one film can be shown in an auditorium at any one time.



Exercise: the multiplex cinema

Let's look at the operator of a multiplex cinema. The multiplex cinema has several auditoriums. Each auditorium has a name, a unique number and contains a number of seats. Each film has a name, was produced in a certain year in a certain language and has a unique code. A film showing takes place on a particular day at a particular time in an auditorium. Obviously only one film can be shown in an auditorium at any one time.




```
create table auditorium(
no int primary key,
seats int,
name varchar(100))
```

```
create table showings(
date date,
time time,
auditorium int references auditorium(no),
movie int references movies(code),
primary key(date,time,auditorium))
```

```
create table movies(
code int primary key,
year int,
name varchar(100),
language varchar(100))
```

Mapping of is-a relationships - discussion

- Every type of mapping has advantages and disadvantages
  - Fewer relations are generally better (fewer joins required)
    - null-value style is good
    - OO style is bad as there are very many relations
    - ER is OK
  - Less space required is generally better
    - OO with only one tuple with exactly the right attributes is thus very good
    - null-value with only one tuple, but that is long
    - ER with many tuples, but only the key is repeated
  - Fast query execution
    - It depends very much on the query!

➔ No clear winner, depends on the application!

Overview of the transformations

ER concept	is mapped to relational concept
Entity type E_i Attributes of E_i Primary key P_i	Relational schema R_i Attributes of R_i Primary key P_i
Relationship type its attributes 1 : n 1 : 1 m : n	Relational schema Attributes: P_1, P_2 more attributes P_2 becomes primary key of the relationship P_1 and P_2 become keys of the relationship $P_1 \cup P_2$ becomes primary key of the relationship
is-a relationship	R_1 gets additional key P_2

- with
  - E_1, E_2 : entity types involved in relationship,
  - P_1, P_2 : their primary keys,
  - 1 : n relationship: E_2 is n side,
  - is-a relationship: E_1 is a more special entity type

Practical approach to database design

- We now know 2 ways to design a database:
  1) From a set of attributes and a set of FDs directly using normalisation (BCNF, 3NF / decomposition, synthesis)
  2) Using an ER model and (semi-)automatic transformation into a relational model (RM)

  ➔ Question: which approach is "the right one"?

- Typical (recommended) approach
  - Prepare an ER model
  - Transform into a relational model
  - If the ER model is good, the RM produced is usually automatically in BCNF!
  - (Automated) check whether the RM is in BCNF (or at least in 3NF)
  - If not: indication of a design flaw in the ER, therefore: go back to the ER model and improve this (very rarely: elimination of the redundancies in the RM)

  ➔ Normal form theory and normalisation is an important tool for designing a good ER model!

Summary

- ER model
  - Entity types
  - Relationships (cardinalities: n:m, 1:n, 1:1)
  - n-ary (degree n) relationships and conversion into binary relationships
  - Recursive relationships
  - Weak (non-identifying) relationships
  - is-a relationship
- Good ER models
- Transformation of an ER model into a relational model
  - Entity types, relationships
  - Merging
  - Mapping of weak (non-identifying) relationships
  - Mapping of is-a relationships